HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN LẬP TRÌNH MẠNG

XÂY DỰNG HỆ THỐNG LUYỆN TẬP VÀ THI ĐẤU LẬP TRÌNH TRỰC TUYẾN THEO MÔ HÌNH CLIENT SERVER

Sinh viên thực hiện : **Phạm Xuân Minh – B23DCCN564**
Nguyễn Trung Kiên – B23DCCN464
Giảng viên hướng dẫn : **Ngô Văn Quyền**
Nhóm lớp : **10**
Nhóm bài tập lớn : **19**

Hà Nội - 2026
Năm học 2026–2027

Nhận xét của giảng viên chấm bài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Điểm số: .....

Điểm chữ: .....

Giảng viên *(ký và ghi rõ họ tên)*: .....

THÔNG TIN ĐỀ TÀI VÀ PHÂN CÔNG

Tên đề tài: Xây dựng hệ thống luyện tập và thi đấu lập trình trực tuyến theo mô hình client server.

Bảng 0.1 Phân công công việc của nhóm

Thành viên	Module và phạm vi phụ trách	Tỷ lệ
Phạm Xuân Minh B23DCCN564	Tài khoản và phân quyền: đăng ký, đăng nhập, hồ sơ, mật khẩu và vai trò. Code Survival và ghép trận: Giao lưu, Normal, Ranked, xác nhận, vòng đấu và loại người chơi. Chấm bài tự động: hàng đợi, Worker, Docker sandbox, testcase, tiến độ và kết quả.	50%
Nguyễn Trung Kiên B23DCCN464	Contest: lịch thi, đăng ký, duyệt tham gia, phòng thi và bảng điểm. Ngân hàng đề và luyện tập: Question Studio, Workspace, tìm kiếm, bản nháp và lịch sử. Thành tích và cộng đồng: EXP, Rating, xếp hạng, lịch sử, bookmark, bình luận, thông báo.	50%

Mỗi thành viên phụ trách giao diện, xử lý phía máy chủ, dữ liệu và kiểm thử của các module được giao. Hai thành viên phối hợp thiết kế giao thức WebSocket, tích hợp hệ thống, triển khai và hoàn thiện báo cáo.

Email liên hệ:

Phạm Xuân Minh — aptmp58@gmail.com.

Nguyễn Trung Kiên — ngtrgkien1011@gmail.com.

MỤC LỤC

THÔNG TIN ĐỀ TÀI VÀ PHÂN CÔNG ..... **i**
DANH MỤC BẢNG ..... **iii**
DANH MỤC HÌNH ..... **iv**
CHƯƠNG 1. GIỚI THIỆU **1**
1.1. Bối cảnh và lý do chọn đề tài 1
1.2. Phát biểu bài toán 1
1.3. Mục tiêu, phạm vi và đóng góp của đề tài 2
CHƯƠNG 2. TỔNG QUAN CÁC NỀN TẢNG LẬP TRÌNH TRỰC TUYẾN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN **3**
2.1. Tổng quan các hệ thống hiện có 3
2.2. Các chức năng phổ biến và những vấn đề đặt ra 3
2.3. Định hướng phát triển chế độ thi đấu Code Survival 4
CHƯƠNG 3. PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG **6**
3.1. Phân tích yêu cầu hệ thống 6
3.2. Công nghệ và kiến trúc tổng thể 7
3.3. Thiết kế giao tiếp mạng và xử lý đồng thời 8
3.4. Thiết kế và cơ chế hoạt động của Code Survival 11
3.5. Thiết kế các chức năng chính của hệ thống 15
3.6. Thiết kế dữ liệu và bảo mật thực thi 17
CHƯƠNG 4. THIẾT LẬP THỰC NGHIỆM VÀ KẾT QUẢ **21**
4.1. Môi trường và phương pháp kiểm thử 21
4.2. Kết quả xây dựng hệ thống 23
4.3. Kết quả triển khai Code Survival 29
4.4. Kết quả kiểm thử và đánh giá kỹ thuật 35
CHƯƠNG 5. THẢO LUẬN, ĐÁNH GIÁ VÀ ĐÓNG GÓP **39**
5.1. Phân tích và đánh giá kết quả 39
5.2. Điểm mới và đóng góp của dự án 40
5.3. Những hạn chế của hệ thống 41
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **43**
6.1. Kết luận 43
6.2. Hướng phát triển 44
TÀI LIỆU THAM KHẢO **45**

DANH MỤC BẢNG

Bảng 0.1 Phân công công việc của nhóm i
Bảng 3.1. Các vai trò và chức năng chính trong hệ thống 6
Bảng 3.2. Các công nghệ chính được sử dụng trong hệ thống 7
Bảng 3.3. Cấu trúc các thông điệp giao tiếp chính 9
Bảng 3.4. Các chế độ thi đấu trong Code Survival 12
Bảng 3.5. Các nhóm dữ liệu chính của hệ thống 17
Bảng 4.1. Môi trường thực nghiệm của hệ thống 21
Bảng 4.2. Tổng hợp kết quả kiểm thử chức năng và giao thức 35
Bảng 4.3. Thời gian phản hồi WebSocket theo số Client đồng thời 36
Bảng 4.4. So sánh kết quả chấm bài với một và hai Worker 36

DANH MỤC HÌNH

Hình 3.1. Sơ đồ kiến trúc tổng thể của hệ thống 8
Hình 3.2. Sơ đồ trình tự giao tiếp và xử lý yêu cầu chấm bài 11
Hình 3.3. Sơ đồ trạng thái và quy trình thi đấu Code Survival 15
Hình 3.4. Sơ đồ quan hệ các nhóm dữ liệu chính của hệ thống 20
Hình 4.1. Giao diện đăng nhập hệ thống 23
Hình 4.2. Giao diện kho bài luyện tập và các bộ lọc 24
Hình 4.3. Giao diện Workspace soạn mã nguồn và nộp bài 24
Hình 4.4. Giao diện lịch sử bài nộp và kết quả chấm tự động 25
Hình 4.5. Giao diện cuộc thi và bảng điểm kết quả 26
Hình 4.6. Giao diện bảng xếp hạng người dùng 27
Hình 4.7. Giao diện Question Studio và cấu hình bài tập 28
Hình 4.8. Giao diện quản trị tài khoản và phân quyền 28
Hình 4.9. Giao diện lựa chọn ba chế độ thi đấu Code Survival 29
Hình 4.10. Giao diện phòng Practice với cấu hình trận đấu 30
Hình 4.11. Giao diện tìm đối thủ ở chế độ Đấu thường 31
Hình 4.12. Danh sách người chơi và trạng thái một trận Code Survival 32
Hình 4.13. Giao diện giải bài, nộp mã nguồn và theo dõi thời gian trong trận 33
Hình 4.14. Bảng trạng thái người chơi tại vòng 2 của Code Survival 34
Hình 4.15. Giao diện kết quả trận và điểm kinh nghiệm được trao 34

CHƯƠNG 1. GIỚI THIỆU

1.1. Bối cảnh và lý do chọn đề tài

Trong quá trình học tập lập trình, việc thực hành giải bài tập, kiểm tra tính đúng đắn của chương trình và tham gia các cuộc thi đóng vai trò quan trọng trong việc rèn luyện kỹ năng. Hiện nay, các hệ thống lập trình trực tuyến cho phép người dùng viết mã, gửi bài và nhận kết quả chấm tự động thông qua Internet. Việc tích hợp các chức năng này trên cùng một hệ thống giúp quá trình luyện tập và đánh giá trở nên thuận tiện, đồng thời giảm sự phụ thuộc vào môi trường thực thi trên máy cá nhân.

Để đáp ứng các hoạt động trên, hệ thống cần có khả năng tiếp nhận yêu cầu từ nhiều người dùng, xử lý bài nộp và trả kết quả chính xác. Mô hình Client–Server phù hợp với bài toán này khi cho phép phân chia nhiệm vụ giữa phía người dùng và máy chủ, đồng thời hỗ trợ quản lý dữ liệu tập trung và tổ chức các hoạt động thi đấu qua mạng.

Từ những yêu cầu thực tế đó, nhóm lựa chọn đề tài **"Xây dựng hệ thống luyện tập và thi đấu lập trình trực tuyến theo mô hình Client–Server"**. Đề tài hướng đến việc vận dụng kiến thức lập trình mạng để xây dựng một hệ thống hỗ trợ luyện tập, chấm bài tự động và tổ chức thi đấu trực tuyến, qua đó nghiên cứu cách giao tiếp giữa các thành phần, xử lý nhiều yêu cầu đồng thời và bảo đảm tính nhất quán của dữ liệu.

1.2. Phát biểu bài toán

Bài toán đặt ra là xây dựng một hệ thống cho phép nhiều người dùng luyện tập và thi đấu lập trình trực tuyến thông qua mô hình Client–Server. Người dùng có thể truy cập ngân hàng bài tập, viết mã nguồn, gửi bài để chấm tự động và theo dõi kết quả. Hệ thống cần quản lý tập trung thông tin người dùng, bài tập, bài nộp và kết quả thi đấu, đồng thời hỗ trợ nhiều phiên làm việc qua mạng.

Về mặt kỹ thuật, hệ thống phải tiếp nhận và xử lý các yêu cầu từ nhiều Client, phân phối tác vụ chấm bài đến các Worker và trả kết quả cho người dùng. Do quá trình thực thi mã nguồn có thể mất thời gian, việc chấm bài cần được xử lý bất đồng bộ để không làm gián đoạn các hoạt động khác. Ngoài ra, hệ thống phải bảo đảm mã nguồn được thực thi trong môi trường cách ly nhằm hạn chế ảnh hưởng đến máy chủ.

Đối với hoạt động thi đấu trực tuyến, hệ thống cần quản lý trạng thái cuộc thi, thời gian nộp bài và kết quả của nhiều người tham gia. Các thay đổi phải được đồng bộ giữa Client và Server, đồng thời xử lý được những tình huống như gửi yêu cầu trùng lặp, mất kết nối hoặc nhiều người dùng thao tác cùng lúc. Đây là những vấn đề cần giải quyết để bảo đảm tính chính xác, nhất quán và công bằng trong quá trình vận hành.

1.3. Mục tiêu, phạm vi và đóng góp của đề tài

Mục tiêu của đề tài là xây dựng một hệ thống luyện tập và thi đấu lập trình trực tuyến theo mô hình Client–Server, cho phép người dùng truy cập ngân hàng bài tập, viết và nộp mã nguồn, nhận kết quả chấm tự động và tham gia các cuộc thi. Hệ thống hướng đến việc tích hợp các hoạt động luyện tập, đánh giá và quản lý kết quả trên một nền tảng thống nhất.

Về phạm vi thực hiện, đề tài tập trung phát triển ứng dụng web với các chức năng quản lý tài khoản, phân quyền người dùng, quản lý bài tập, chấm bài tự động, tổ chức thi đấu, bảng xếp hạng và các tiện ích hỗ trợ học tập. Phía máy chủ chịu trách nhiệm xử lý yêu cầu, quản lý dữ liệu và điều phối các tác vụ chấm bài đến Worker. Việc thực thi mã nguồn được thực hiện trong môi trường cách ly nhằm hạn chế rủi ro đối với hệ thống.

Về đóng góp, đề tài đã xây dựng và tích hợp các thành phần của một hệ thống luyện tập và thi đấu lập trình trực tuyến, bao gồm giao diện người dùng, máy chủ xử lý nghiệp vụ, cơ sở dữ liệu và các Worker thực thi mã nguồn. Hệ thống sử dụng WebSocket để trao đổi dữ liệu, kết hợp cơ chế xử lý bất đồng bộ và quản lý trạng thái tập trung nhằm hỗ trợ nhiều người dùng tham gia các hoạt động luyện tập và thi đấu qua mạng.

CHƯƠNG 2. TỔNG QUAN CÁC NỀN TẢNG LẬP TRÌNH TRỰC TUYẾN VÀ ĐỊNH HƯỚNG PHÁT TRIỂN

2.1. Tổng quan các hệ thống hiện có

Các nền tảng lập trình trực tuyến là những hệ thống hỗ trợ người dùng thực hành, kiểm tra và đánh giá kỹ năng lập trình thông qua môi trường web. Các hệ thống này thường cung cấp ngân hàng bài tập, trình soạn thảo mã nguồn và công cụ chấm tự động, cho phép tiếp nhận lời giải, thực thi chương trình và đánh giá kết quả dựa trên các bộ dữ liệu kiểm thử. Ngoài hoạt động luyện tập cá nhân, nhiều nền tảng còn hỗ trợ tổ chức cuộc thi, quản lý kết quả, xếp hạng và theo dõi thành tích của người tham gia.

Một số nền tảng tiêu biểu hiện nay gồm LeetCode, HackerRank và Codeforces. LeetCode tập trung vào các bài toán thuật toán và cấu trúc dữ liệu, phục vụ luyện tập và chuẩn bị phỏng vấn kỹ thuật. HackerRank cung cấp bài tập theo ngôn ngữ và lĩnh vực lập trình, kết hợp các công cụ đánh giá kỹ năng và tổ chức thử thách trực tuyến. Trong khi đó, Codeforces chú trọng hoạt động thi đấu thuật toán thông qua các cuộc thi, hệ thống chấm bài tự động, bảng xếp hạng và điểm đánh giá năng lực người tham gia. Mặc dù có định hướng riêng, các nền tảng trên đều đáp ứng những nhu cầu cơ bản về luyện tập và thi đấu lập trình trực tuyến.

Qua khảo sát, có thể nhận thấy các chức năng như quản lý bài tập, chấm mã nguồn tự động, tổ chức cuộc thi và theo dõi thành tích đã được triển khai phổ biến. Đây là cơ sở để xác định các chức năng nền tảng, đồng thời xem xét những yêu cầu đặt ra đối với việc phát triển các hình thức luyện tập, giao lưu và thi đấu có tính tương tác giữa nhiều người dùng.

2.2. Các chức năng phổ biến và những vấn đề đặt ra

Các nền tảng lập trình trực tuyến hiện nay đã cung cấp tương đối đầy đủ các chức năng phục vụ quá trình học tập và đánh giá kỹ năng lập trình. Người dùng có thể lựa chọn bài tập theo chủ đề và độ khó, viết mã nguồn, gửi lời giải để chấm tự động và

theo dõi lịch sử bài nộp. Bên cạnh đó, các chức năng như tổ chức cuộc thi, cập nhật bảng xếp hạng và thống kê thành tích cũng được triển khai nhằm hỗ trợ việc đánh giá năng lực và tạo động lực luyện tập.

Tuy nhiên, hoạt động trên các nền tảng này vẫn chủ yếu tập trung vào quá trình giải bài cá nhân và đánh giá kết quả thông qua các cuộc thi. Mặc dù một số hệ thống đã hỗ trợ thảo luận, tương tác cộng đồng và thi đấu trực tuyến, các hình thức giao lưu trực tiếp giữa những người chơi trong một trận đấu nhóm nhỏ chưa phải là định hướng trọng tâm. Trong khi đó, nhu cầu luyện tập cùng bạn bè, cạnh tranh trực tiếp và theo dõi tiến trình thi đấu của những người tham gia đặt ra yêu cầu phát triển các hình thức tổ chức linh hoạt, có tính tương tác cao hơn.

Việc mở rộng hoạt động luyện tập sang các hình thức thi đấu nhiều người cũng đặt ra những yêu cầu kỹ thuật đối với hệ thống. Ngoài khả năng tiếp nhận và chấm bài tự động, máy chủ cần quản lý đồng thời nhiều người tham gia, đồng bộ trạng thái trận đấu, kiểm soát thời gian và cập nhật kết quả theo thời gian thực. Đặc biệt, do các bài nộp có thể được xử lý bất đồng bộ, hệ thống cần bảo đảm tính nhất quán và công bằng khi xác định kết quả, đồng thời xử lý được các tình huống mất kết nối hoặc nhiều người dùng thao tác cùng lúc.

Những vấn đề trên là cơ sở để nghiên cứu và phát triển hình thức thi đấu lập trình có sự tương tác trực tiếp giữa nhiều người chơi, kết hợp hoạt động giải bài với cơ chế cạnh tranh theo từng vòng. Hướng phát triển này được nhóm lựa chọn triển khai thông qua chế độ Code Survival, trình bày tại mục 2.3.

2.3. Định hướng phát triển chế độ thi đấu Code Survival

Code Survival là chế độ thi đấu lập trình theo hình thức sinh tồn nhiều vòng, được phát triển nhằm tăng cường khả năng giao lưu, kết nối và cạnh tranh trực tiếp giữa những người chơi. Trong một trận đấu, các thành viên cùng tham gia giải bài lập trình và trải qua những vòng loại liên tiếp để xác định người chiến thắng. Hình thức này hướng đến việc tạo ra môi trường luyện tập có tính tương tác, kết hợp giữa rèn luyện kỹ năng giải thuật và thi đấu trực tuyến.

Code Survival được xây dựng với ba chế độ gồm Giao lưu, Đấu thường và Đấu xếp hạng. Chế độ Giao lưu cho phép tạo phòng tùy chỉnh từ 2 đến 10 người, hỗ trợ thiết lập độ khó và thời gian thi đấu, phù hợp với hoạt động luyện tập cùng bạn bè. Chế độ Đấu thường tổ chức các trận đấu gồm 5 người thông qua cơ chế ghép trận tự động và trao điểm kinh nghiệm theo thành tích. Chế độ Đấu xếp hạng cũng tổ chức trận đấu 5 người nhưng sử dụng điểm xếp hạng để ghép đối thủ và điều chỉnh thành tích sau khi trận đấu kết thúc.

Trong mỗi vòng thi, những người chơi còn lại cùng giải một bài lập trình trong thời gian giới hạn. Hệ thống tiếp nhận và chấm bài tự động, sau đó xác định những người được tiếp tục thi đấu dựa trên kết quả của vòng hiện tại. Người chơi không đáp ứng điều kiện sẽ bị loại, trong khi những người còn lại tiếp tục tham gia các vòng tiếp theo cho đến khi xác định được người chiến thắng. Các thành viên có thể theo dõi thời gian, trạng thái bài nộp, kết quả từng vòng và tiến trình của trận đấu. Người đã bị loại vẫn có thể theo dõi các vòng thi tiếp theo.

Để bảo đảm tính nhất quán trong quá trình thi đấu, trạng thái trận đấu được quản lý tập trung tại máy chủ và đồng bộ đến các người chơi theo thời gian thực. Việc xác định kết quả dựa trên bài nộp hợp lệ và thời điểm máy chủ tiếp nhận, hạn chế ảnh hưởng của sự chênh lệch thời gian chấm bài. Hệ thống cũng hỗ trợ xử lý các tình huống đồng hạng, mất kết nối và lỗi chấm bài nhằm bảo đảm kết quả thi đấu được xác định theo những quy tắc thống nhất.

Code Survival góp phần đa dạng hóa hình thức luyện tập và thi đấu lập trình thông qua cơ chế sinh tồn nhiều vòng, ghép trận theo chế độ và tương tác trực tiếp giữa những người tham gia. Việc kết hợp hoạt động giải bài với quá trình loại người chơi và cập nhật kết quả theo thời gian thực tạo ra môi trường thi đấu có tính cạnh tranh, đồng thời đáp ứng nhu cầu giao lưu và rèn luyện kỹ năng lập trình.

CHƯƠNG 3. PHƯƠNG PHÁP VÀ THIẾT KẾ HỆ THỐNG

3.1. Phân tích yêu cầu hệ thống

Hệ thống được xây dựng nhằm đáp ứng nhu cầu luyện tập, chấm bài tự động và thi đấu lập trình trực tuyến theo mô hình Client–Server. Các yêu cầu được xác định dựa trên hoạt động của người dùng và các điều kiện cần thiết để hệ thống vận hành. Trong đó, yêu cầu chức năng mô tả những nghiệp vụ hệ thống cần thực hiện, còn yêu cầu phi chức năng tập trung vào khả năng giao tiếp mạng, xử lý đồng thời, tính nhất quán dữ liệu và an toàn thực thi.

Yêu cầu chức năng: Hệ thống cần hỗ trợ quản lý tài khoản, xác thực và phân quyền người dùng; cung cấp ngân hàng bài tập, trình soạn thảo mã nguồn, chức năng nộp bài và chấm tự động. Người dùng có thể theo dõi kết quả, lịch sử bài nộp, tham gia các cuộc thi và xem bảng xếp hạng. Hệ thống đồng thời hỗ trợ tạo và quản lý bài tập, tổ chức cuộc thi, quản lý nội dung và các hoạt động tương tác cộng đồng. Đối với Code Survival, các chức năng cần đáp ứng gồm tạo phòng, ghép trận, quản lý người tham gia, tổ chức các vòng thi và xác định kết quả theo quy tắc của từng chế độ.

Bảng 3.1. Các vai trò và chức năng chính trong hệ thống

Vai trò	Chức năng chính
ADMIN (Quản trị viên)	Quản lý tài khoản, phân quyền người dùng, quản lý bài tập, cuộc thi và thông báo hệ thống.
AUTHOR (Người tạo đề)	Tạo, chỉnh sửa và quản lý bài tập; tạo và tổ chức các cuộc thi lập trình.
CODER (Người luyện tập)	Luyện tập, nộp bài, tham gia cuộc thi và các chế độ thi đấu, theo dõi kết quả và bảng xếp hạng.

Yêu cầu phi chức năng: Hệ thống cần duy trì khả năng phục vụ nhiều kết nối Client và xử lý các yêu cầu đồng thời mà không làm sai lệch dữ liệu. Quá trình chấm bài phải được tổ chức bất đồng bộ để hạn chế ảnh hưởng đến hoạt động tiếp nhận và xử lý yêu cầu của máy chủ. Mã nguồn do người dùng cung cấp phải được thực thi

trong môi trường cách ly, có giới hạn tài nguyên và quyền truy cập nhằm bảo đảm an toàn cho hệ thống.

Đối với giao tiếp mạng, các yêu cầu cần được xác thực, kiểm tra tính hợp lệ và kiểm soát việc xử lý trùng lặp. Hệ thống phải hỗ trợ khôi phục kết nối và đồng bộ những thay đổi trạng thái cần thiết khi Client bị gián đoạn. Trong các hoạt động thi đấu nhiều người, trạng thái trận đấu, thời điểm tiếp nhận bài nộp và kết quả chấm phải được quản lý thống nhất nhằm bảo đảm tính chính xác, nhất quán và công bằng giữa những người tham gia.

3.2. Công nghệ và kiến trúc tổng thể

Hệ thống được thiết kế theo mô hình Client–Server, gồm các thành phần chính: Client, Server, cơ sở dữ liệu và Worker chấm bài. Mỗi thành phần đảm nhận một nhóm nhiệm vụ riêng, trong đó Server đóng vai trò trung tâm tiếp nhận yêu cầu, xử lý nghiệp vụ và quản lý dữ liệu. Quá trình thực thi mã nguồn được tách khỏi Server nhằm hạn chế ảnh hưởng của các tác vụ chấm bài đến hoạt động chung của hệ thống.

Bảng 3.2. Các công nghệ chính được sử dụng trong hệ thống

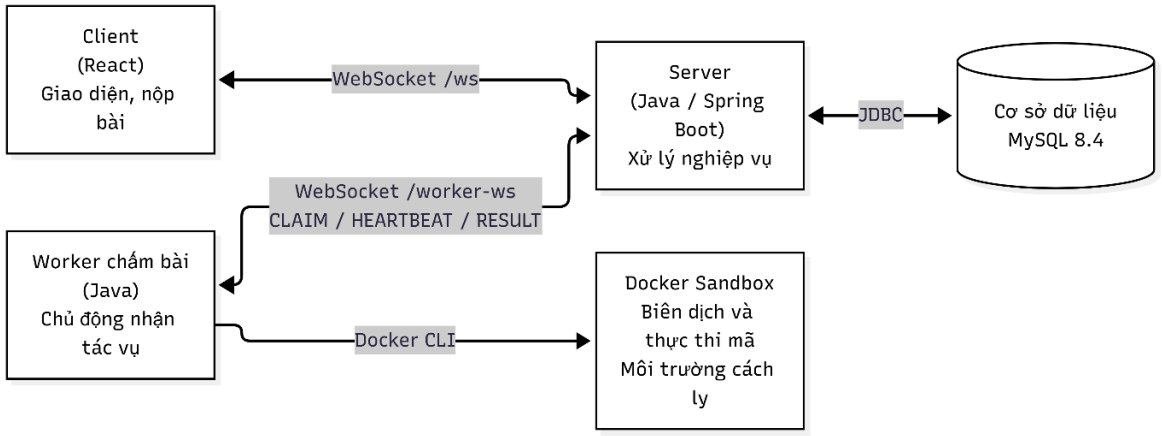
Thành phần	Công nghệ	Vai trò
Frontend	React 19, Vite, Tailwind CSS, CodeMirror	Xây dựng giao diện, soạn thảo mã nguồn và tương tác với máy chủ
Backend	Java 21, Spring Boot, JDBC	Xử lý nghiệp vụ, quản lý kết nối và truy xuất dữ liệu
Giao tiếp mạng	WebSocket, JSON	Trao đổi yêu cầu, phản hồi và cập nhật trạng thái
Cơ sở dữ liệu	MySQL 8.4, Flyway	Lưu trữ dữ liệu và quản lý cấu trúc cơ sở dữ liệu
Hệ thống chấm bài	Java Worker, Docker	Tiếp nhận tác vụ chấm và thực thi mã nguồn trong môi trường cách ly
Triển khai	Docker Compose	Quản lý và kết nối các dịch vụ của hệ thống

Phía Client được phát triển bằng React, cung cấp giao diện để người dùng thực hiện các thao tác luyện tập, nộp bài và tham gia thi đấu. Trình soạn thảo CodeMirror được tích hợp để hỗ trợ viết mã nguồn trực tiếp trên trình duyệt. Client trao đổi dữ liệu với Server thông qua kết nối WebSocket, trong khi phần giao diện sau khi biên dịch được cung cấp dưới dạng tài nguyên tĩnh.

Phía Server được xây dựng bằng Java và Spring Boot, tổ chức theo kiến trúc phân lớp gồm Controller, Service và Repository. Controller tiếp nhận yêu cầu từ Client, Service xử lý các quy tắc nghiệp vụ, còn Repository thực hiện truy xuất dữ liệu thông qua JDBC. Hệ thống sử dụng MySQL để lưu trữ thông tin người dùng, bài tập, bài nộp, cuộc thi và trạng thái thi đấu. Công cụ Flyway được sử dụng để quản lý và cập nhật cấu trúc cơ sở dữ liệu.

Đối với chức năng chấm bài tự động, Server tiếp nhận bài nộp và lưu tác vụ chấm vào cơ sở dữ liệu. Worker hoạt động độc lập, kết nối với Server qua kênh WebSocket riêng và chủ động yêu cầu nhận tác vụ từ hàng đợi. Sau khi nhận bài, Worker sử dụng Docker Sandbox để biên dịch và thực thi mã nguồn trong môi trường cách ly. Kết quả xử lý được gửi về Server để kiểm tra, tổng hợp và lưu trữ trước khi cập nhật cho Client. Worker không truy cập trực tiếp vào cơ sở dữ liệu, giúp phân tách trách nhiệm giữa quá trình quản lý nghiệp vụ và thực thi chương trình.

Các thành phần được triển khai và kết nối thông qua Docker Compose, bao gồm dịch vụ Server, MySQL và Worker cùng môi trường Docker phục vụ chấm bài. Cách tổ chức này cho phép quản lý các dịch vụ độc lập, đồng thời duy trì sự thống nhất trong quá trình trao đổi dữ liệu giữa các thành phần.



Hình 3.1. Sơ đồ kiến trúc tổng thể của hệ thống

3.3. Thiết kế giao tiếp mạng và xử lý đồng thời

Hệ thống sử dụng WebSocket để duy trì kết nối hai chiều giữa Client và Server, cho phép trao đổi yêu cầu, phản hồi và cập nhật trạng thái trong quá trình hoạt động.

Hai kênh giao tiếp được thiết lập gồm /ws dành cho người dùng và /worker-ws dành cho các Worker chấm bài. Dữ liệu được truyền dưới định dạng JSON, với cấu trúc thông điệp thống nhất nhằm hỗ trợ kiểm tra và xử lý yêu cầu tại máy chủ.

Giao thức phía Client sử dụng ba loại thông điệp chính gồm yêu cầu thực hiện lệnh (Command), phản hồi xác nhận (ACK) và thông báo sự kiện (EVENT). Mỗi yêu cầu được gắn mã định danh requestId để xác định thao tác tương ứng. Server kiểm tra tính hợp lệ của thông điệp, thực hiện xử lý nghiệp vụ và trả phản hồi xác nhận sau khi giao dịch hoàn tất. Các thay đổi trạng thái cần thông báo được lưu thành sự kiện và gửi đến Client thông qua kết nối WebSocket.

Bảng 3.3. Cấu trúc các thông điệp giao tiếp chính

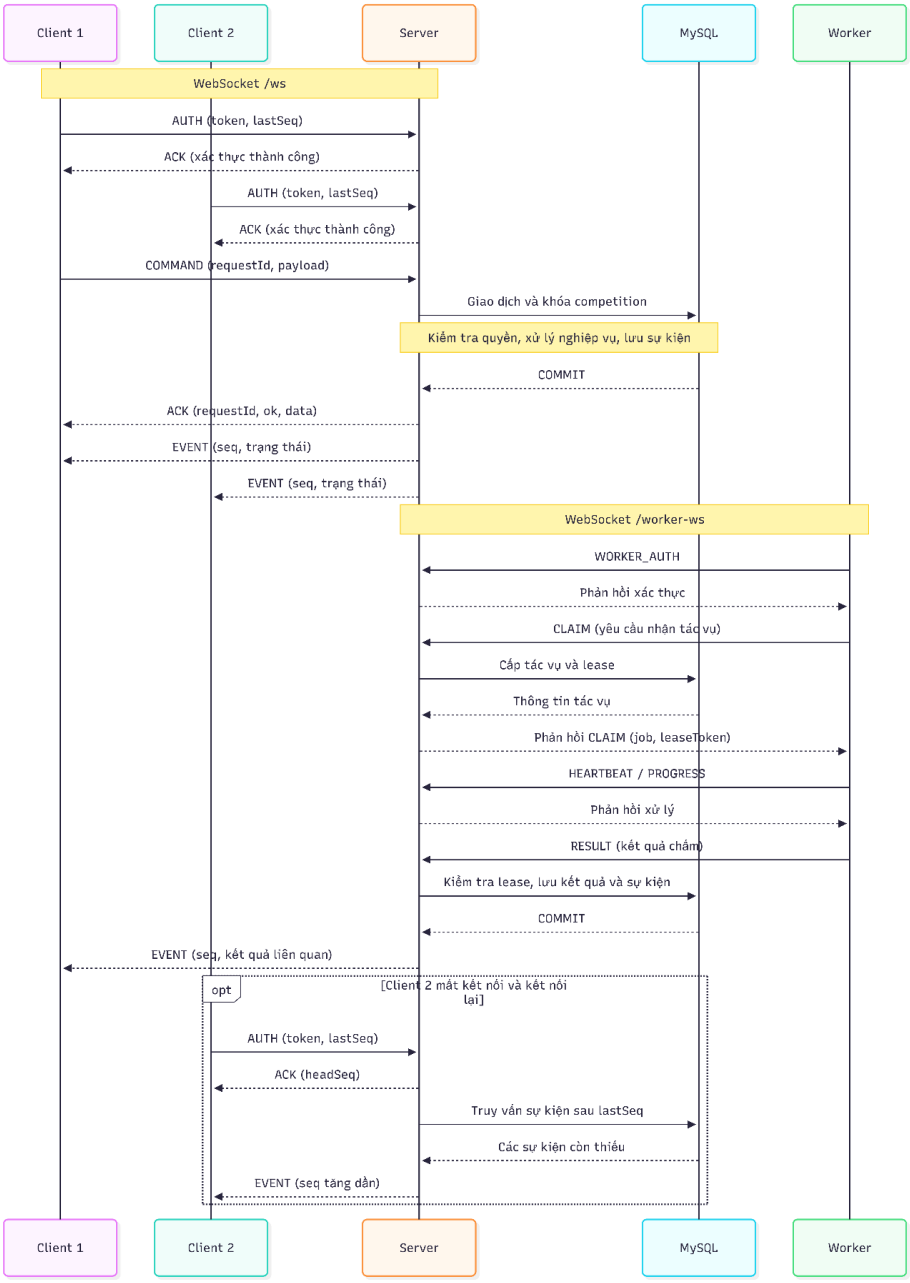
Thông điệp	Trường dữ liệu chính	Chức năng
Yêu cầu (Command)	v, type, requestId, payload	Gửi yêu cầu từ Client đến Server
Phản hồi (ACK)	v, type, requestId, command, ok, data/error	Xác nhận kết quả xử lý yêu cầu
Sự kiện (EVENT)	v, type, seq, event, data	Thông báo thay đổi trạng thái đến Client
Yêu cầu Worker	v, type, requestId, payload	Xác thực, nhận tác vụ và gửi tiến độ, kết quả chấm

Đối với quá trình chấm bài, Server tiếp nhận và lưu trữ bài nộp trước khi đưa tác vụ vào hàng đợi. Worker kết nối qua kênh riêng, xác thực bằng token và chủ động gửi lệnh CLAIM để nhận tác vụ. Trong quá trình thực hiện, Worker sử dụng thông điệp HEARTBEAT để duy trì quyền xử lý, PROGRESS để cập nhật tiến độ và RESULT để gửi kết quả về Server. Cơ chế cấp quyền xử lý có thời hạn (lease) được sử dụng nhằm kiểm soát trạng thái tác vụ và từ chối kết quả từ Worker khi quyền xử lý không còn hợp lệ. Việc tổ chức chấm bài bất đồng bộ cho phép Server tiếp tục phục vụ các yêu cầu khác trong thời gian chương trình của người dùng được thực thi.

Để kiểm soát các thao tác đồng thời, hệ thống sử dụng giao dịch cơ sở dữ liệu kết hợp cơ chế khóa. Những thao tác làm thay đổi dữ liệu được thực hiện trong giao dịch, sử dụng khóa competition để tuần tự hóa quá trình cập nhật trạng thái và kết quả. Các thao tác đọc được xử lý trong giao dịch chỉ đọc nhằm duy trì dữ liệu nhất quán mà không cần sử dụng khóa ghi chung. Đối với yêu cầu thay đổi dữ liệu, Server lưu

requestId, dấu vân tay nội dung và phản hồi đã xử lý. Khi Client gửi lại cùng một yêu cầu, hệ thống có thể trả kết quả đã lưu thay vì thực hiện lặp lại thao tác, đồng thời từ chối trường hợp tái sử dụng requestId với nội dung khác.

Khi kết nối bị gián đoạn, Client có thể thiết lập lại kết nối và xác thực phiên làm việc. Các sự kiện được đánh số thứ tự bằng trường seq, cho phép Client sử dụng lastSeq để tiếp nhận những cập nhật chưa nhận được. Hệ thống đồng thời kiểm tra định dạng thông điệp, giới hạn tần suất yêu cầu và xác thực quyền truy cập nhằm hạn chế các thao tác không hợp lệ. Những cơ chế này hỗ trợ duy trì tính nhất quán của dữ liệu và trạng thái thi đấu trong môi trường có nhiều Client hoạt động đồng thời.



Hình 3.2. Sơ đồ trình tự giao tiếp và xử lý yêu cầu chấm bài

3.4. Thiết kế và cơ chế hoạt động của Code Survival

Code Survival được thiết kế theo hình thức thi đấu lập trình sinh tồn nhiều vòng, trong đó những người tham gia cùng giải bài và cạnh tranh để giành quyền đi tiếp. Server quản lý tập trung quá trình tổ chức trận đấu, trạng thái người chơi, kết quả từng vòng và thành tích cuối cùng. Chế độ này sử dụng chung ngân hàng bài tập và

hệ thống chấm tự động, đồng thời bổ sung các cơ chế ghép trận, loại người chơi và xử lý những tình huống phát sinh trong thi đấu nhiều người.

Thiết kế các chế độ thi đấu: Code Survival hỗ trợ ba chế độ gồm Giao lưu, Đấu thường và Đấu xếp hạng. Các chế độ sử dụng chung cơ chế xử lý trận đấu nhưng khác nhau về hình thức tham gia, số lượng người chơi và cách ghi nhận thành tích.

Bảng 3.4. Các chế độ thi đấu trong Code Survival

Chế độ	Số người	Hình thức tham gia	Ghi nhận thành tích
Giao lưu	2–10	Tạo phòng tùy chỉnh, tham gia bằng mã phòng hoặc danh sách phòng	Không cộng EXP và không thay đổi Rating
Đấu thường	5	Ghép trận tự động, ưu tiên cấp độ gần nhau	Cộng EXP theo thứ hạng
Đấu xếp hạng	5	Ghép trận tự động theo Survival Rating	Cộng EXP và cập nhật Rating

Cơ chế tạo phòng và ghép trận: Ở chế độ Giao lưu, người tạo phòng có thể thiết lập số lượng thành viên, mức độ khó của bài tập và thời gian thi đấu. Những người chơi khác tham gia thông qua mã phòng hoặc danh sách phòng đang mở. Chủ phòng được thay đổi cấu hình khi trận chưa bắt đầu; cấu hình được khóa khi trận chuyển sang giai đoạn đếm ngược.

Đối với Đấu thường và Đấu xếp hạng, Server quản lý hàng đợi ghép trận trong cơ sở dữ liệu và tự động lựa chọn nhóm năm người. Chế độ Đấu thường ưu tiên những người có cấp độ gần nhau trong 10 giây đầu, sau đó ưu tiên thời gian chờ. Chế độ Đấu xếp hạng sử dụng độ chênh lệch Rating, với phạm vi tìm kiếm được mở rộng dần theo thời gian chờ. Khi tìm được nhóm phù hợp, hệ thống gửi yêu cầu xác nhận tham gia trong thời hạn 20 giây. Trận đấu chỉ được khởi tạo khi đủ năm người xác nhận; trường hợp từ chối hoặc hết hạn được xử lý để tránh bắt đầu trận không hợp lệ.

Cơ chế quản lý trạng thái và tổ chức vòng thi: Trận đấu được điều khiển thông qua các trạng thái chính gồm LOBBY (phòng chờ), COUNTDOWN (đếm ngược), ROUND_ACTIVE (đang thi đấu), ROUND_GRADING (chờ chấm), ROUND_RESULT (công bố kết quả) và FINISHED (kết thúc). Việc chuyển trạng thái do Server quyết định dựa trên thời gian và kết quả xử lý của từng vòng.

Trước khi bắt đầu, hệ thống kiểm tra và cố định tập bài đủ điều kiện chấm, bao gồm các bài đã được xác thực và có bộ testcase hợp lệ. Trong mỗi vòng, tất cả người chơi còn quyền thi đấu nhận cùng một bài tập và phải hoàn thành trong thời gian quy định. Mỗi người được nộp tối đa hai bài chính thức. Khi hết thời gian hoặc tất cả người chơi còn lại đã nộp bài, vòng chuyển sang giai đoạn chờ chấm để tổng hợp kết quả trước khi xác định người đi tiếp.

Cơ chế đánh giá và loại người chơi: Kết quả vòng đấu được xác định dựa trên trạng thái chấm, thời điểm Server tiếp nhận bài và số lượt nộp sai. Người có bài đạt Accepted (AC) được ưu tiên xét đi tiếp. Trong nhóm đạt AC, hệ thống tính thời gian hiệu lực bằng khoảng thời gian từ lúc bắt đầu vòng đến khi Server nhận bài AC, cộng thêm 10 giây cho mỗi lượt nộp sai trước đó. Người có thời gian hiệu lực thấp hơn được xếp trên. Đối với bài chưa đạt AC, hệ thống ghi nhận điểm số cao nhất để phục vụ hiển thị kết quả, nhưng không xét đi tiếp trong vòng loại thông thường.

Khi còn trên hai người, số suất đi tiếp được xác định theo quy tắc $S(n) = \max(2, \lfloor 0{,}7n \rfloor)$, trong đó n là số người được xét ở vòng hiện tại. Khi chỉ còn hai người, hệ thống xác định một người chiến thắng. Vì vậy, đối với trận năm người, tiến trình loại thông thường là $\mathbf{5 \rightarrow 3 \rightarrow 2 \rightarrow 1}$, nếu các vòng đều xác định được kết quả hợp lệ và không phát sinh tình huống đặc biệt.

Đồng bộ trạng thái và bảo đảm tính công bằng: Trong quá trình thi đấu, Server lưu trữ trạng thái phòng, thời hạn từng vòng, bài nộp và kết quả chấm. Những thay đổi được gửi đến các Client liên quan thông qua WebSocket để người chơi theo dõi cùng một trạng thái trận đấu. Client chỉ hiển thị thông tin nhận được, còn quyền xác định kết quả và chuyển vòng thuộc về Server.

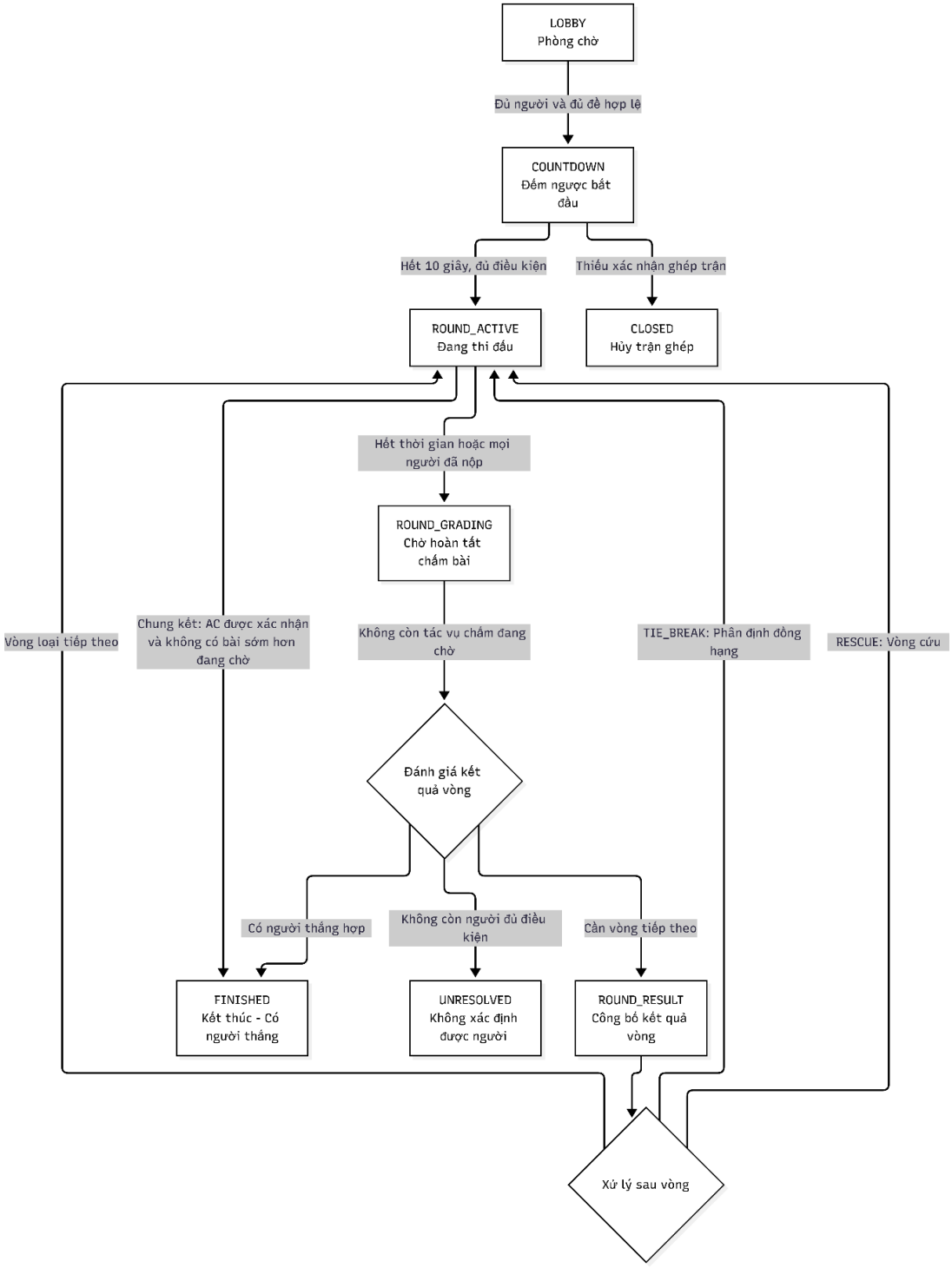
Do các bài nộp được Worker xử lý bất đồng bộ, thứ tự hoàn thành chấm không nhất thiết trùng với thứ tự nộp bài. Vì vậy, hệ thống sử dụng thời điểm Server tiếp nhận bài làm cơ sở tính thời gian thi đấu. Đặc biệt, trong vòng quyết định người chiến thắng, một bài AC đã chấm xong không được ưu tiên kết thúc trận nếu vẫn còn bài nộp sớm hơn đang chờ chấm và có khả năng ảnh hưởng đến kết quả. Các thao tác cập nhật trạng thái được thực hiện trong giao dịch có kiểm soát đồng thời nhằm hạn chế sai lệch khi nhiều người chơi thao tác gần như cùng lúc.

Xử lý các tình huống đặc biệt: Khi xảy ra đồng hạng tại vị trí quyết định quyền đi tiếp, hệ thống tổ chức vòng phân định TIE_BREAK cho những người đang hòa, trong khi người đã chắc suất được giữ nguyên kết quả. Nếu vòng đấu gặp lỗi hạ tầng chấm bài hoặc bị quản trị viên xác định sử dụng đề không hợp lệ, hệ thống có thể tổ chức vòng cứu hộ RESCUE thay vì loại người chơi do sự cố.

Đối với trường hợp mất kết nối, trạng thái trận đấu vẫn được lưu tại Server để hỗ trợ khôi phục khi người chơi kết nối lại. Người chơi không duy trì trạng thái hiện diện quá thời hạn cho phép có thể bị ghi nhận bỏ cuộc. Nếu không còn người đủ điều kiện hoặc không thể xác định người chiến thắng hợp lệ, trận đấu chuyển sang trạng thái UNRESOLVED (chưa phân định kết quả) và không thực hiện trao thưởng.

Cơ chế tính điểm và cập nhật thành tích: Khi trận đấu kết thúc hợp lệ, thứ hạng được xác định theo tiến trình tồn tại qua các vòng loại; những người bị loại cùng giai đoạn có thể được xếp đồng hạng. Chế độ Giao lưu không làm thay đổi điểm kinh nghiệm (EXP) hoặc Rating. Đối với Đấu thường và Đấu xếp hạng, EXP được trao theo thứ hạng, với mức thưởng từ 100 đến 20 EXP cho năm vị trí trong trường hợp không có đồng hạng.

Riêng chế độ Đấu xếp hạng, hệ thống sử dụng cơ chế Elo nhiều người, tính mức thay đổi Rating dựa trên điểm trước trận và kết quả so sánh giữa từng cặp người chơi. Điểm Rating tăng có thể được áp dụng hệ số theo bậc xếp hạng, trong khi điểm bị trừ không sử dụng hệ số này. Việc lưu kết quả và cập nhật phần thưởng được thực hiện trong giao dịch cơ sở dữ liệu, kết hợp kiểm tra lịch sử để tránh trao thưởng nhiều lần cho cùng một trận.



Hình 3.3. Sơ đồ trạng thái và quy trình thi đấu Code Survival

3.5. Thiết kế các chức năng chính của hệ thống

Hệ thống được tổ chức thành các nhóm chức năng gồm quản lý người dùng, quản lý bài tập, luyện tập lập trình, chấm bài tự động, tổ chức cuộc thi và các tiện ích hỗ

trợ. Các chức năng được thực hiện thông qua giao diện web, trong khi Server chịu trách nhiệm kiểm tra quyền truy cập, xử lý nghiệp vụ và lưu trữ dữ liệu.

Quản lý người dùng và phân quyền: Hệ thống hỗ trợ đăng ký, đăng nhập, đăng xuất, cập nhật hồ sơ và thay đổi mật khẩu. Người dùng được phân thành ba vai trò: ADMIN, AUTHOR và CODER. ADMIN có quyền quản lý tài khoản, phân quyền và trạng thái hoạt động của người dùng; AUTHOR được tạo, chỉnh sửa bài tập do mình quản lý và tổ chức cuộc thi; CODER thực hiện luyện tập và tham gia thi đấu. Các thao tác được kiểm tra quyền tại Server. Khi quyền hoặc trạng thái tài khoản thay đổi, những phiên đăng nhập liên quan được thu hồi nhằm ngăn việc tiếp tục sử dụng quyền truy cập cũ.

Quản lý bài tập: Ngân hàng bài tập hỗ trợ tìm kiếm theo tên hoặc mã bài, lọc theo danh mục, độ khó và trạng thái hoàn thành. Đối với người tạo đề, công cụ Question Studio cho phép thiết lập nội dung bài toán, ngôn ngữ hỗ trợ, đáp án mẫu, bộ testcase và các giới hạn thực thi. Bài tập được quản lý theo hai trạng thái: bản nháp (Draft) và đã công bố (Ready). Trước khi sử dụng để chấm, cấu hình bài tập được Server kiểm tra về tính hợp lệ của dữ liệu và các điều kiện thực thi. Hệ thống đồng thời hỗ trợ kiểm tra đáp án mẫu để người tạo đề đánh giá cấu hình chấm.

Luyện tập và nộp bài: Người dùng lựa chọn bài tập từ ngân hàng đề và thực hiện giải bài trên Workspace tích hợp trình soạn thảo CodeMirror. Hệ thống hỗ trợ C++11, C++17, C++20 và Python 3.11; ngôn ngữ được lựa chọn phải thuộc danh sách cho phép của bài tập. Workspace cung cấp chức năng lưu bản nháp, kiểm tra trước khi nộp và nộp bài chính thức. Khi tiếp nhận bài nộp, Server kiểm tra dữ liệu đầu vào, ghi nhận mã nguồn, ngôn ngữ, thời điểm tiếp nhận và trạng thái xử lý. Sau khi chấm hoàn tất, kết quả được lưu để người dùng xem chi tiết, tra cứu lịch sử và theo dõi tiến độ giải bài.

Tổ chức cuộc thi và xếp hạng: Người có quyền tổ chức có thể tạo Contest, lựa chọn bài tập, thiết lập thời gian bắt đầu, kết thúc và hình thức đăng ký. Cuộc thi hỗ trợ tham gia trực tiếp hoặc yêu cầu người tổ chức phê duyệt. Trong thời gian diễn ra Contest, người tham gia được truy cập các bài tập thuộc cuộc thi và gửi lời giải hợp lệ. Hệ thống tổng hợp kết quả chấm để xác định điểm số, số bài giải đúng và thời gian

phạt, từ đó sắp xếp thứ hạng theo điểm giảm dần và thời gian phạt tăng dần. Đối với Contest được cấu hình tính xếp hạng, kết quả cuối cùng còn được sử dụng để cập nhật điểm Elo của người tham gia.

Các chức năng hỗ trợ và quản trị: Hệ thống cung cấp chức năng đánh dấu bài tập yêu thích, bình luận theo từng bài, lưu bản nháp mã nguồn và theo dõi lịch sử bài nộp. Bảng xếp hạng và hồ sơ cá nhân hiển thị các thông tin thành tích như số bài giải đúng, điểm Elo, điểm kinh nghiệm và tiến trình luyện tập. Ngoài ra, ADMIN có thể quản lý tài khoản, đăng thông báo và tra cứu nhật ký hoạt động để phục vụ công tác quản trị hệ thống.

3.6. Thiết kế dữ liệu và bảo mật thực thi

Hệ thống sử dụng MySQL 8.4 để lưu trữ và quản lý dữ liệu tập trung. Cấu trúc cơ sở dữ liệu được khởi tạo và quản lý bằng Flyway, gồm 27 bảng được tổ chức theo các nhóm nghiệp vụ. Thiết kế kết hợp dữ liệu quan hệ với các trường lưu nội dung JSON nhằm đáp ứng yêu cầu quản lý tài khoản, ngân hàng bài tập, kết quả chấm và trạng thái thi đấu. Các bảng sử dụng khóa chính, khóa ngoại, ràng buộc duy nhất và chỉ mục để bảo đảm tính toàn vẹn dữ liệu, đồng thời hỗ trợ truy vấn theo từng chức năng.

Bảng 3.5. Các nhóm dữ liệu chính của hệ thống

Nhóm dữ liệu	Các bảng tiêu biểu	Chức năng
Người dùng	users, sessions, progression	Quản lý tài khoản, phiên đăng nhập, điểm kinh nghiệm và xếp hạng
Bài tập	problems, categories, problem_numbers	Lưu trữ nội dung, phân loại và quản lý mã bài tập
Chấm bài	submissions, submission_sequence, solved	Quản lý bài nộp, thứ tự tiếp nhận, trạng thái chấm và kết quả
Cuộc thi và phòng đấu	contests, contest_problems, rooms, room_members, room_problem_bank	Quản lý cuộc thi, phòng thi đấu, thành viên và danh sách bài tập
Ghép trận và thành tích	match_queue, match_results, reward_events	Lưu hàng đợi ghép trận, kết quả thi đấu và lịch sử phần thưởng

Chức năng hỗ trợ	bookmarks, comments, announcements, drafts	Quản lý bài đánh dấu, bình luận, thông báo và bản nháp
Giao tiếp và kiểm soát	commands, events, audit, transaction_locks	Lưu yêu cầu, sự kiện, nhật ký và hỗ trợ kiểm soát giao dịch

Đối với ngân hàng bài tập, bảng problems lưu thông tin cơ bản và nội dung cấu hình đề dưới dạng JSON, bao gồm mô tả bài toán, đáp án mẫu, testcase và giới hạn thực thi. Các quan hệ giữa bài tập với danh mục, cuộc thi và phòng đấu được quản lý thông qua những bảng riêng. Khi tiếp nhận bài nộp, Server lưu mã nguồn, ngôn ngữ, thời điểm tiếp nhận, thứ tự nộp và bản chụp cấu hình đề (*snapshot*) trong bảng submissions. Cơ chế này giúp quá trình chấm sử dụng đúng dữ liệu đề tại thời điểm nộp, không bị ảnh hưởng bởi những thay đổi nội dung sau đó.

Đối với Code Survival, dữ liệu trận đấu được quản lý thông qua bảng rooms, kết hợp với room_members và room_problem_bank để xác định thành viên và tập bài được sử dụng. Bảng match_queue lưu trạng thái hàng đợi phục vụ ghép trận tự động, trong khi match_results ghi nhận thứ hạng và thay đổi điểm sau trận. Thông tin EXP và Survival Rating được quản lý trong progression; lịch sử trao thưởng được lưu tại reward_events với khóa duy nhất nhằm hạn chế việc cộng thưởng trùng lặp. Việc kết hợp dữ liệu JSON với các bảng quan hệ giúp lưu được trạng thái trận đấu có cấu trúc linh hoạt, đồng thời hỗ trợ truy vấn thành viên, bài tập và lịch sử thi đấu.

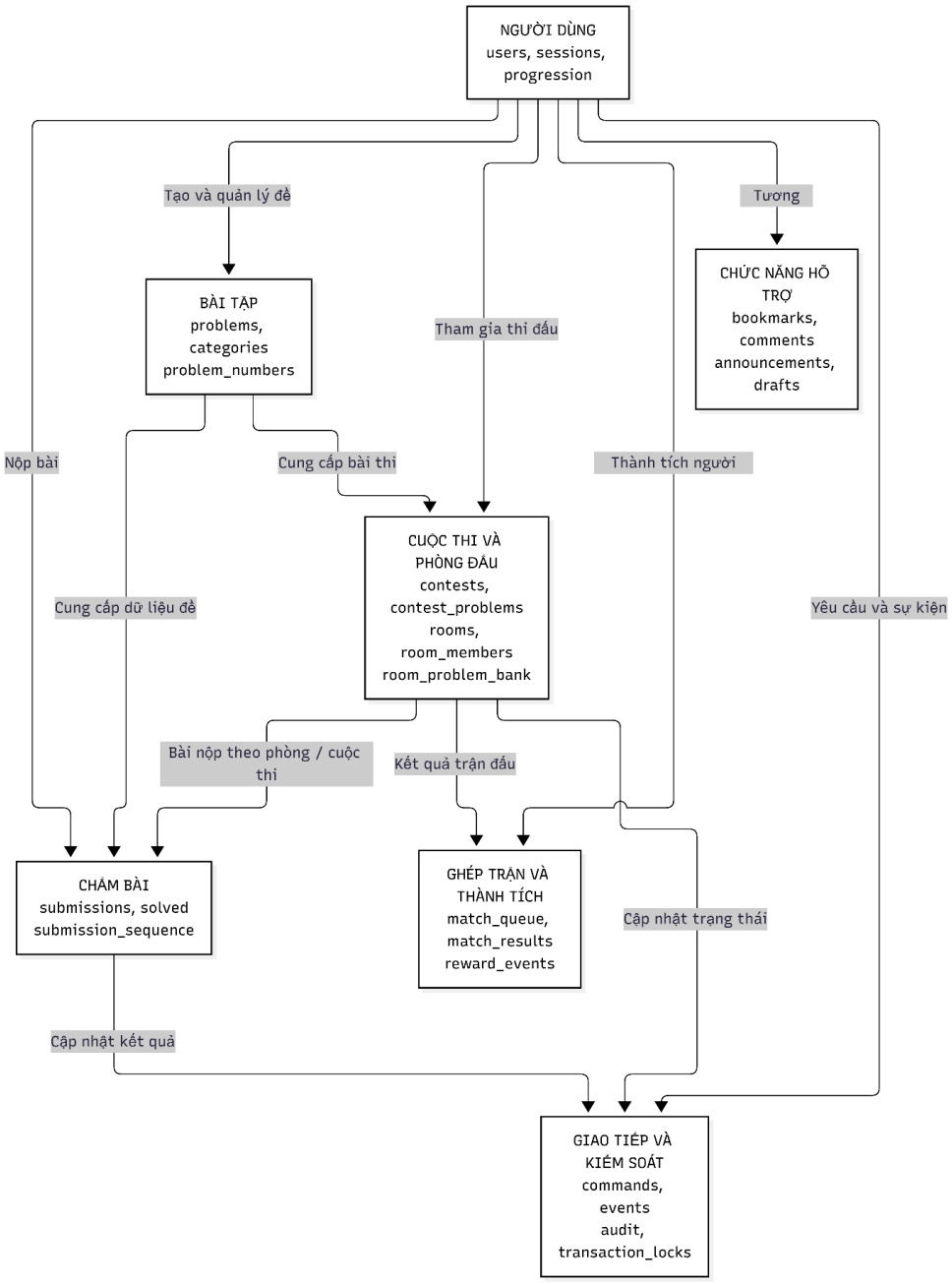
Để duy trì tính nhất quán, các thao tác cập nhật dữ liệu nghiệp vụ được thực hiện trong giao dịch cơ sở dữ liệu. Hệ thống sử dụng bảng transaction_locks để hỗ trợ kiểm soát những thao tác đồng thời; bảng commands lưu mã yêu cầu, dấu vân tay nội dung và phản hồi đã xử lý nhằm hạn chế thực hiện trùng lặp. Các sự kiện được lưu trong events, còn audit ghi nhận những thao tác cần theo dõi. Những cơ chế này hỗ trợ bảo toàn trạng thái bài nộp, trận đấu và kết quả khi có nhiều yêu cầu được xử lý gần như đồng thời.

Bảo mật dữ liệu và kiểm soát truy cập: Mật khẩu người dùng được băm bằng BCrypt thay vì lưu dưới dạng văn bản thuần. Token phiên đăng nhập được tạo ngẫu nhiên và chỉ lưu giá trị băm SHA-256 trong cơ sở dữ liệu. Các thao tác truy cập, chỉnh sửa dữ liệu và quản trị được Server kiểm tra theo trạng thái tài khoản và vai trò ADMIN, AUTHOR hoặc CODER. Hệ thống đồng thời kiểm tra tính hợp lệ của dữ

liệu đầu vào và giới hạn quyền truy cập đối với những nội dung không được phép công khai.

Đối với bộ chấm tự động, hệ thống phân biệt testcase công khai và testcase ẩn, đồng thời kiểm soát mức độ hiển thị phản hồi chấm bài. Worker chỉ nhận mã nguồn, cấu hình thực thi và dữ liệu kiểm thử cần thiết; đáp án kỳ vọng được giữ tại Server để đối chiếu sau khi Worker trả kết quả. Cách phân chia này hạn chế việc tiết lộ dữ liệu chấm và tách biệt trách nhiệm thực thi chương trình với trách nhiệm đánh giá kết quả.

Bảo mật môi trường thực thi: Mã nguồn do người dùng cung cấp được biên dịch và thực thi trong Docker Sandbox thay vì chạy trực tiếp trong tiến trình Server. Worker khởi tạo container riêng cho từng testcase với các giới hạn về CPU, bộ nhớ, số tiến trình, thời gian thực thi và dung lượng đầu ra. Container bị vô hiệu hóa kết nối mạng, sử dụng hệ thống tệp chỉ đọc, chạy bằng tài khoản không có quyền quản trị và được loại bỏ các quyền hệ thống không cần thiết. Sau khi thực thi, container được dọn dẹp để hạn chế tồn dư tài nguyên. Trường hợp Docker không khả dụng, Worker báo lỗi hạ tầng thay vì chuyển sang thực thi mã nguồn trực tiếp trên máy chủ.



Hình 3.4. Sơ đồ quan hệ các nhóm dữ liệu chính của hệ thống

CHƯƠNG 4. THIẾT LẬP THỰC NGHIỆM VÀ KẾT QUẢ

4.1. Môi trường và phương pháp kiểm thử

Quá trình kiểm thử được tổ chức nhằm đánh giá tính đúng đắn của các chức năng, khả năng giao tiếp giữa Client và Server, hiệu quả xử lý chấm bài bất đồng bộ và cơ chế xác định kết quả thi đấu trong Code Survival. Các phép thử kết hợp kiểm thử tự động, kiểm thử tích hợp và đo hiệu năng với số lượng Client, Worker khác nhau. Hệ thống được triển khai trên môi trường Docker với cơ sở dữ liệu thử nghiệm riêng nhằm bảo đảm các thao tác kiểm thử không ảnh hưởng đến dữ liệu đang sử dụng.

Bảng 4.1. Môi trường thực nghiệm của hệ thống

Thành phần	Cấu hình sử dụng
Máy thực nghiệm	Windows 11 Pro, Intel Core i5-12450H (8 nhân, 12 luồng), RAM 15,71 GiB
Nền tảng triển khai	Docker Desktop, Docker Compose, WSL2
Server và cơ sở dữ liệu	Java 21, Spring Boot 3.5.7, MySQL 8.4
Worker chấm bài	Java Worker, Docker Sandbox; khảo sát cấu hình một và hai Worker
Công cụ kiểm thử	JUnit, H2, Node.js, JSDOM, Playwright và Python

Kiểm thử chức năng và giao thức: Các nghiệp vụ Backend được kiểm tra bằng JUnit kết hợp H2 ở chế độ tương thích MySQL, trong khi Frontend được kiểm tra bằng Node.js và JSDOM. Các kịch bản Playwright được thực hiện trên trình duyệt Microsoft Edge nhằm kiểm tra những thao tác chính trong quá trình luyện tập, quản lý nội dung và sử dụng Code Survival. Đối với hệ thống chấm bài, các phép thử được thực hiện bằng Worker và Docker Sandbox với các ngôn ngữ C++11, C++17, C++20,

Python 3.11, bao gồm những trường hợp AC, WA, CE và TLE. Giao thức WebSocket được kiểm tra thông qua các tình huống xác thực, phân quyền, khôi phục kết nối, kiểm tra phiên bản thông điệp và xử lý yêu cầu trùng lặp. Ngoài ra, ngân hàng 320 bài tập với 1.600 testcase được kiểm chứng riêng bằng lời giải tham chiếu nhằm đánh giá tính hợp lệ của dữ liệu đề và bộ kiểm thử.

Đo giao tiếp WebSocket: Phép đo sử dụng thông điệp PING ở tầng ứng dụng để xác định thời gian phản hồi giữa Client và Server qua kênh /ws. Các kịch bản lần lượt sử dụng 1, 10 và 30 Client đồng thời; mỗi Client gửi 50 yêu cầu và mỗi mức tải được thực hiện sáu lượt đo. Thời gian phản hồi RTT được tính từ thời điểm gửi PING đến khi nhận được ACK có requestId tương ứng, sau đó tổng hợp các chỉ số trung vị (p50), phân vị 95% (p95) và thời gian phản hồi lớn nhất. Phép đo sử dụng kết nối chưa xác thực, được thực hiện từ máy thực nghiệm đến Server trong Docker trên cùng máy.

Kiểm thử chấm bài bất đồng bộ: Khả năng xử lý bài nộp được khảo sát với cấu hình một và hai Worker. Ở mỗi cấu hình, số Client đồng thời lần lượt là 1, 5 và 10; mỗi mức tải được lặp lại ba lần. Các bài nộp sử dụng cùng chương trình Python 3.11 tính tổng hai số, với năm testcase cho mỗi bài. Kết quả được đối chiếu trực tiếp với MySQL nhằm kiểm tra trạng thái chấm, điểm số, số testcase hoàn thành và tính duy nhất của bài nộp. Hai chỉ số đánh giá chính gồm thông lượng xử lý, xác định bằng số bài hoàn tất trên một giây, và thời gian xử lý đầu cuối (E2E), tính từ khi Client gửi bài đến khi nhận được kết quả cuối cùng. Thời gian E2E bao gồm thời gian chờ hàng đợi, thực thi và khoảng thăm dò kết quả của Client.

Kiểm tra tính công bằng trong Code Survival: Cơ chế xác định người đi tiếp được đánh giá thông qua mô phỏng 10.000 lượt, mỗi lượt gồm 10 người chơi và bảy suất đi tiếp. Mỗi người được giả định có một bài đạt Accepted (AC); thời điểm tiếp nhận và độ trễ Worker được tạo ngẫu nhiên với hạt giống cố định. Hai phương pháp được so sánh là xếp hạng theo thời điểm hoàn thành chấm và xếp hạng theo thời điểm Server tiếp nhận bài. Tập bảy bài được tiếp nhận sớm nhất được sử dụng làm chuẩn đối chiếu; tỷ lệ sai lệch được xác định bằng số lượt lựa chọn khác tập tham chiếu chia cho tổng số lượt mô phỏng. Phép thử tập trung đánh giá ảnh hưởng của độ trễ Worker

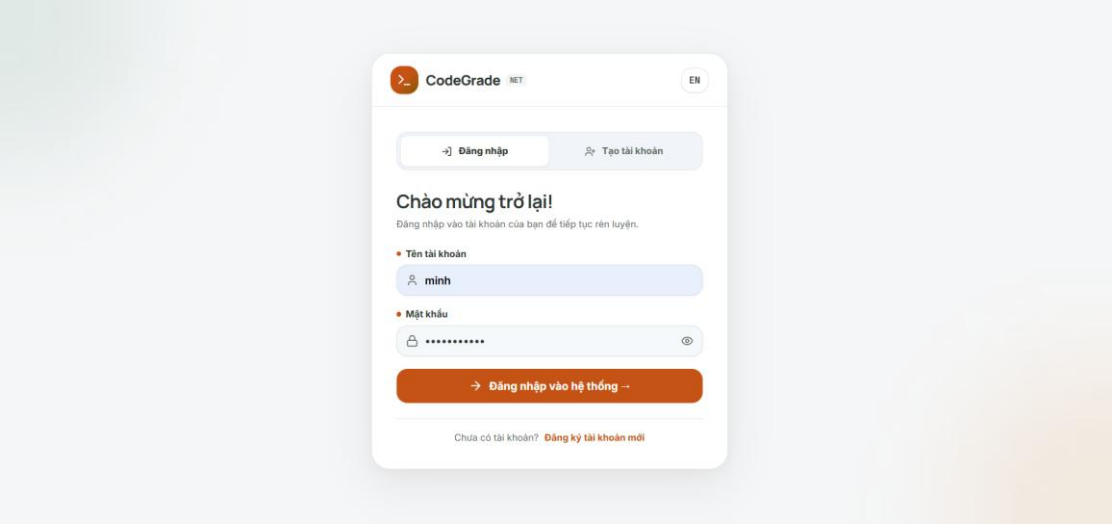
đến kết quả vòng loại, chưa bao quát toàn bộ tình huống của một trận Code Survival thực tế.

Các kết quả kiểm thử được tổng hợp từ phản hồi giao thức, dữ liệu thực nghiệm và bản ghi cơ sở dữ liệu. Các phép đo được thực hiện trong phạm vi tải hữu hạn trên cùng một máy, làm cơ sở cho việc phân tích kết quả và đánh giá kỹ thuật tại mục 4.4.

4.2. Kết quả xây dựng hệ thống

Hệ thống đã triển khai các chức năng phục vụ xác thực người dùng, luyện tập lập trình, chấm bài tự động, tổ chức cuộc thi và quản lý nội dung. Giao diện được xây dựng trên nền tảng web, kết nối với Server thông qua WebSocket. Các chức năng được phân chia theo ba vai trò CODER, AUTHOR và ADMIN.

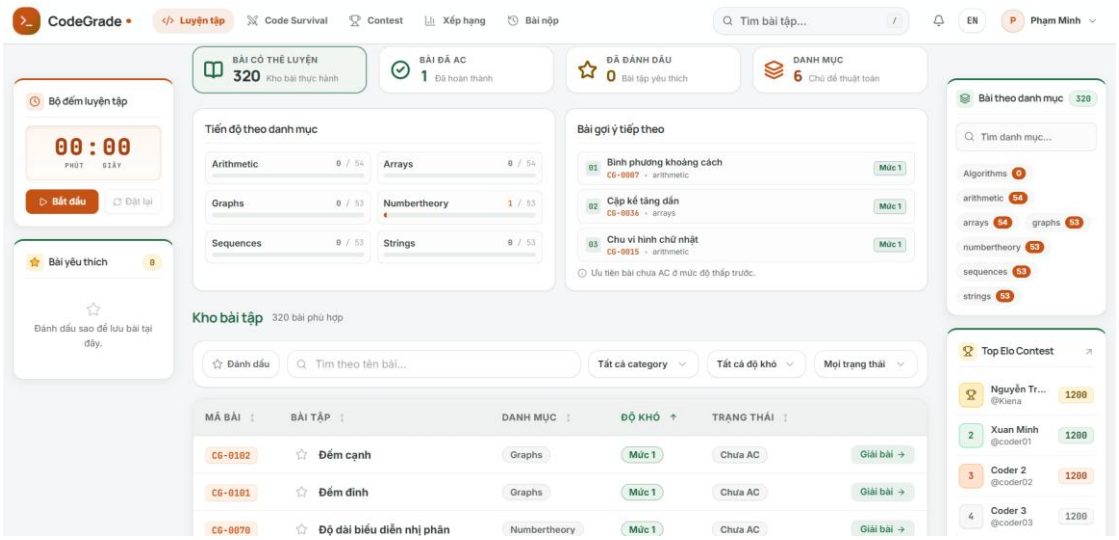
Chức năng đăng nhập và xác thực người dùng: Hệ thống cung cấp giao diện đăng nhập và đăng ký tài khoản. Người dùng đăng nhập bằng tên tài khoản và mật khẩu; sau khi xác thực thành công, Client nhận token phiên làm việc và sử dụng lệnh AUTH để xác thực kết nối WebSocket. Hệ thống duy trì trạng thái đăng nhập, hỗ trợ đăng xuất và kiểm tra quyền truy cập tại Server. Ngoài ra, người dùng có thể xem hồ sơ, cập nhật thông tin cá nhân và thay đổi mật khẩu.



Hình 4.1. Giao diện đăng nhập hệ thống

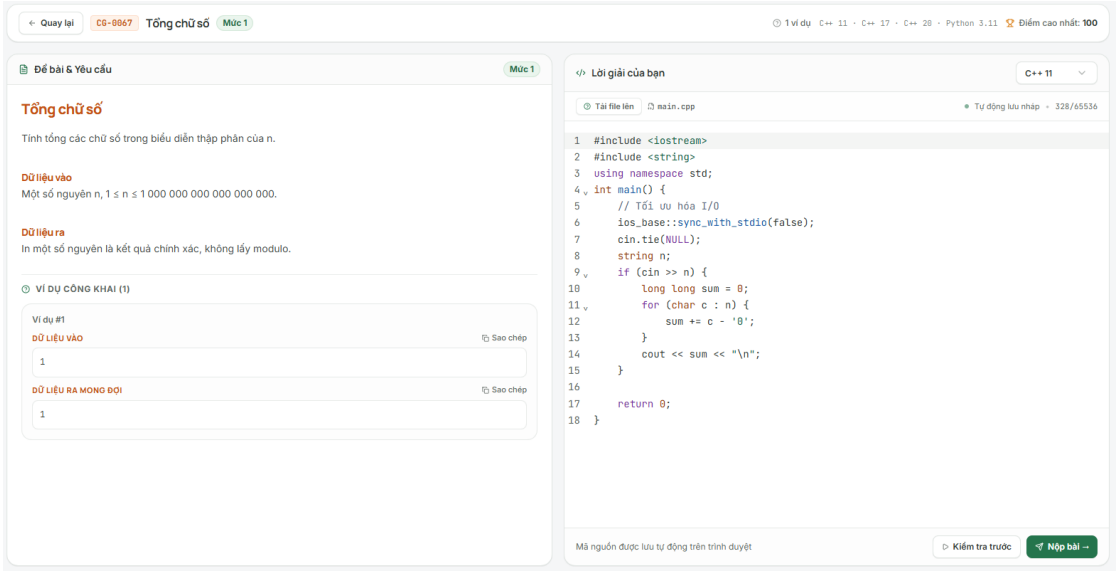
Chức năng quản lý kho bài luyện tập: Hệ thống cung cấp danh sách bài tập được phân loại theo chủ đề và độ khó, giúp người dùng lựa chọn nội dung luyện tập phù hợp. Giao diện hỗ trợ tìm kiếm theo tên hoặc mã bài, lọc theo danh mục, độ khó

và trạng thái hoàn thành. Người dùng cũng có thể đánh dấu bài yêu thích, theo dõi số bài đã giải và tiến độ luyện tập theo từng danh mục.



Hình 4.2. Giao diện kho bài luyện tập và các bộ lọc

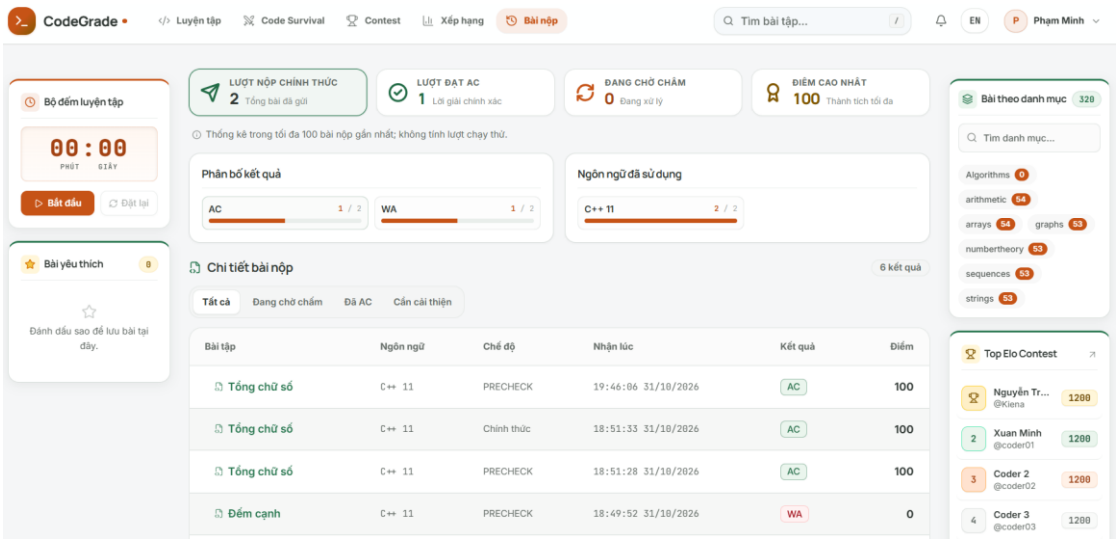
Chức năng soạn thảo mã nguồn và nộp bài: Khi chọn bài tập, người dùng được chuyển đến Workspace với nội dung đề, mô tả đầu vào/đầu ra, ví dụ và trình soạn thảo CodeMirror. Hệ thống hỗ trợ C++11, C++17, C++20 và Python 3.11. Người dùng có thể viết mã trực tiếp, lựa chọn ngôn ngữ, tải mã nguồn từ tệp và tiếp tục chỉnh sửa bản nháp đã lưu. Ngoài thao tác nộp bài chính thức, Workspace còn hỗ trợ kiểm tra trước khi nộp hoặc chạy ví dụ tùy theo ngữ cảnh và cấu hình bài tập.



Hình 4.3. Giao diện Workspace soạn mã nguồn và nộp bài

Chức năng chấm bài tự động và quản lý kết quả: Sau khi tiếp nhận bài nộp, Server lưu tác vụ để Worker thực hiện biên dịch và chạy chương trình trên các testcase. Kết quả thực thi được gửi về Server để đối chiếu với đáp án kỳ vọng và xác định kết quả chấm. Trạng thái xử lý được cập nhật về Client, cho phép người dùng theo dõi quá trình chấm. Hệ thống hỗ trợ các kết quả như Accepted (AC), Wrong Answer (WA), Compilation Error (CE), Time Limit Exceeded (TLE) và các lỗi thực thi khác.

Trang Bài nộp hiển thị lịch sử chấm với các thông tin về thời điểm tiếp nhận, ngôn ngữ, chế độ nộp, điểm số và kết quả. Người dùng có thể lọc, phân trang và xem chi tiết từng bài nộp. Phản hồi testcase được hiển thị theo cấu hình của đề, bảo đảm không công khai dữ liệu kiểm thử ẩn. Ngoài ra, hệ thống còn hỗ trợ xem lịch sử theo từng bài, bảng xếp hạng của bài và bình luận trao đổi giữa người học.

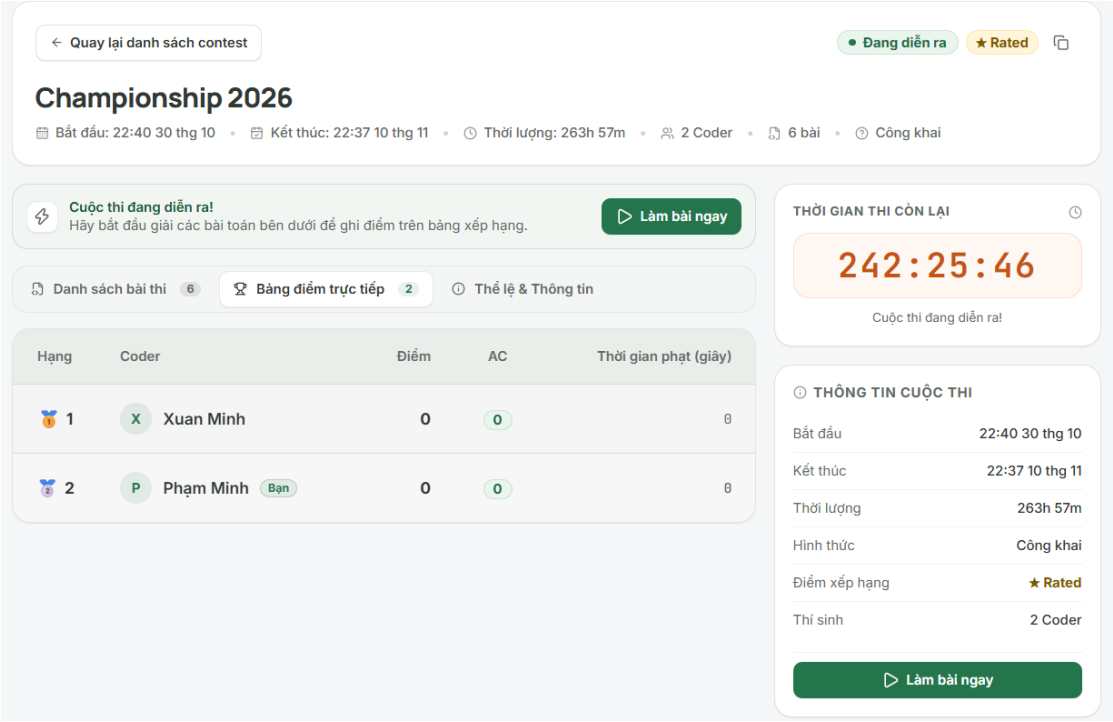


Hình 4.4. Giao diện lịch sử bài nộp và kết quả chấm tự động

Chức năng tổ chức cuộc thi lập trình: Hệ thống cho phép tạo và quản lý Contest với các thông tin như tên cuộc thi, danh sách bài tập, thời gian bắt đầu, thời gian kết thúc và hình thức tham gia. Người tổ chức có thể thiết lập cuộc thi công khai hoặc yêu cầu phê duyệt người tham gia. Trong thời gian diễn ra cuộc thi, thí sinh có thể truy cập các bài tập và gửi lời giải để hệ thống chấm.

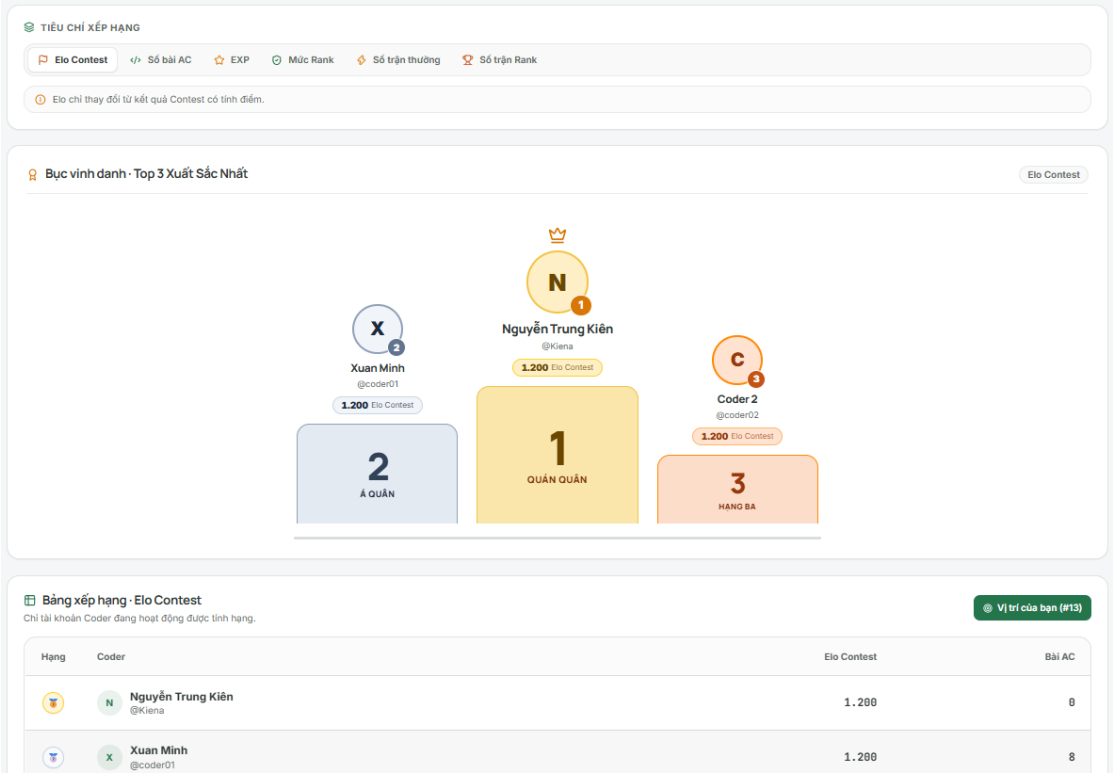
Bảng điểm cuộc thi tổng hợp số bài giải đúng, điểm số và thời gian phạt. Thứ hạng được xác định theo điểm số giảm dần, sau đó xét thời gian phạt tăng dần. Đối

với Contest có tính Elo, kết quả được sử dụng để cập nhật điểm xếp hạng của người tham gia.



Hình 4.5. Giao diện cuộc thi và bảng điểm kết quả

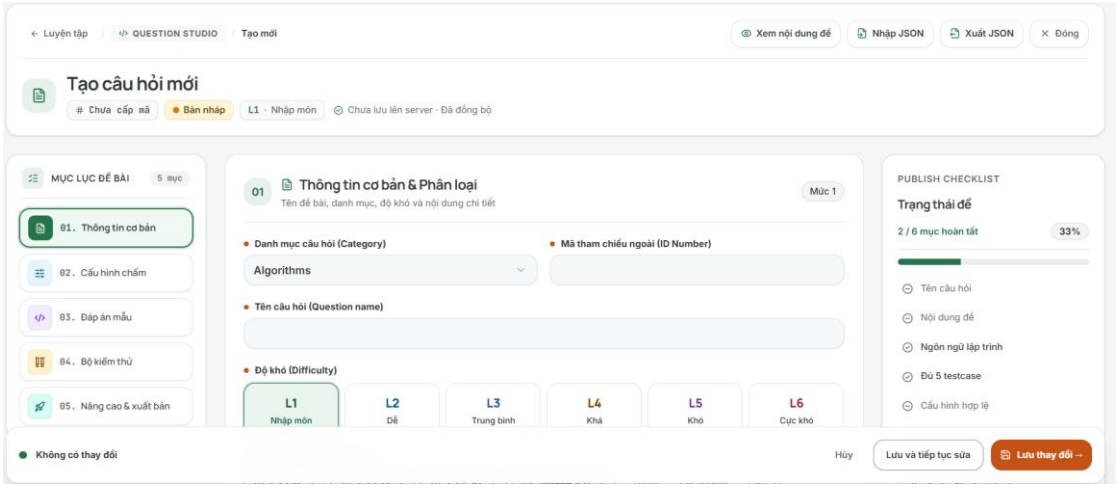
Chức năng bảng xếp hạng và thống kê thành tích: Hệ thống cung cấp bảng xếp hạng người dùng theo các tiêu chí gồm Elo Contest, số bài AC, điểm kinh nghiệm (EXP), mức Rank, số trận Normal và số trận Ranked. Giao diện hiển thị những người có thành tích cao và vị trí của tài khoản đang sử dụng. Các thông tin này được kết hợp với hồ sơ cá nhân để hỗ trợ theo dõi quá trình luyện tập và thi đấu.



Hình 4.6. Giao diện bảng xếp hạng người dùng

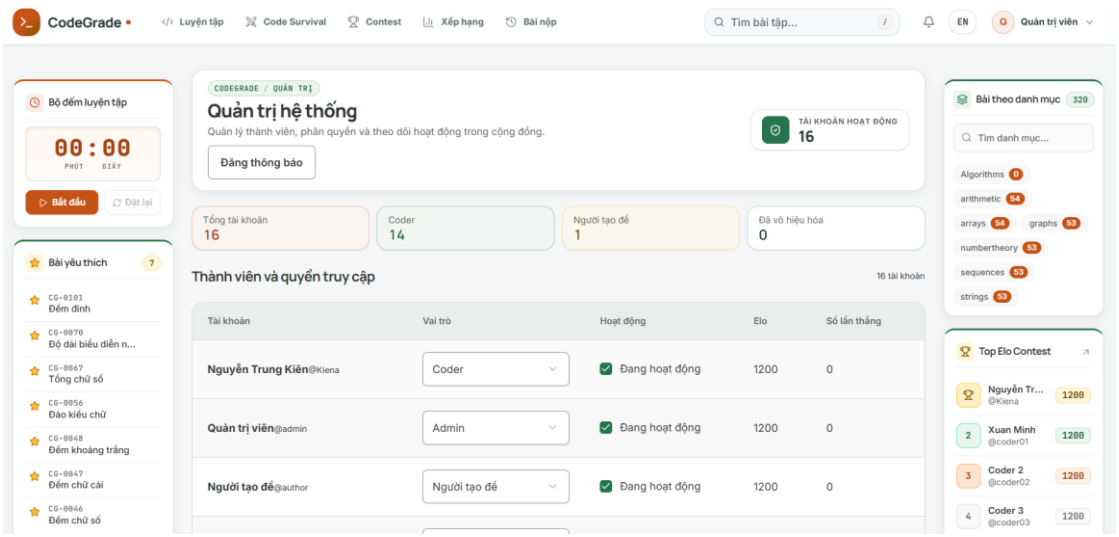
Chức năng soạn đề và quản lý bài tập: Hệ thống cung cấp Question Studio cho vai trò AUTHOR và ADMIN, hỗ trợ tạo mới, chỉnh sửa và quản lý bài tập lập trình. Người tạo đề có thể thiết lập nội dung, độ khó, ngôn ngữ hỗ trợ, đáp án mẫu, mã nguồn khởi tạo và bộ testcase. Công cụ cho phép cấu hình giới hạn thời gian, bộ nhớ, chế độ hiển thị testcase và nhập/xuất dữ liệu dưới định dạng JSON.

Bài tập được quản lý theo trạng thái bản nháp (Draft) hoặc công bố (Ready). Trước khi công bố, người tạo đề có thể sử dụng chức năng kiểm tra đáp án mẫu để đánh giá cấu hình chấm. Việc tổ chức ngân hàng bài còn được hỗ trợ thông qua chức năng tạo và chỉnh sửa danh mục.



Hình 4.7. Giao diện Question Studio và cấu hình bài tập

Chức năng quản trị hệ thống: Giao diện quản trị dành cho ADMIN hỗ trợ quản lý danh sách tài khoản, phân quyền CODER, AUTHOR, ADMIN và thay đổi trạng thái hoạt động của người dùng. Những thao tác quản trị được kiểm tra quyền tại Server nhằm bảo đảm chỉ tài khoản hợp lệ mới có thể thực hiện. Ngoài ra, hệ thống cung cấp nhật ký hoạt động để theo dõi các thao tác và chức năng đăng thông báo phục vụ việc quản lý, vận hành.



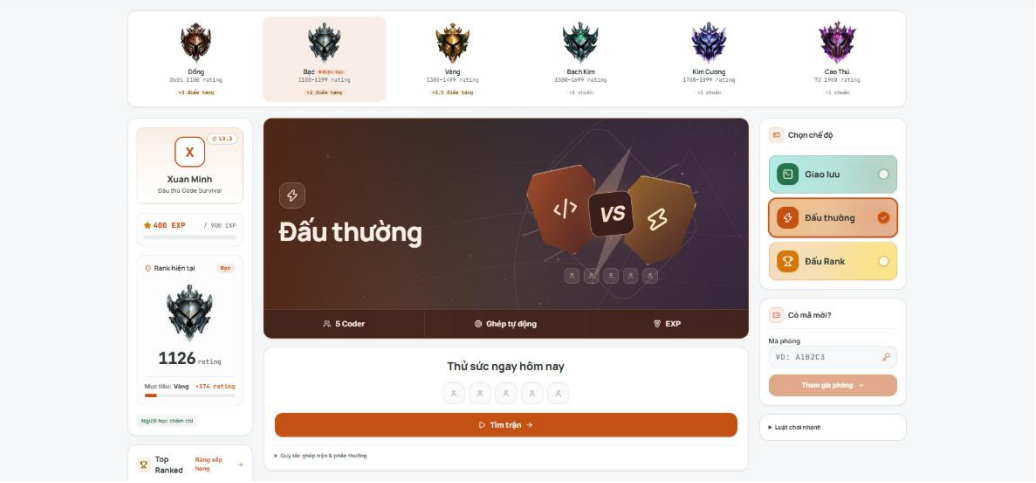
Hình 4.8. Giao diện quản trị tài khoản và phân quyền

Các chức năng trên đã hình thành một hệ thống hỗ trợ tương đối đầy đủ quá trình luyện tập, chấm bài, tổ chức cuộc thi và quản lý người dùng. Trên nền tảng đó, chế độ Code Survival được tích hợp nhằm mở rộng khả năng thi đấu lập trình nhiều người theo thời gian thực, được trình bày riêng tại mục 4.3.

4.3. Kết quả triển khai Code Survival

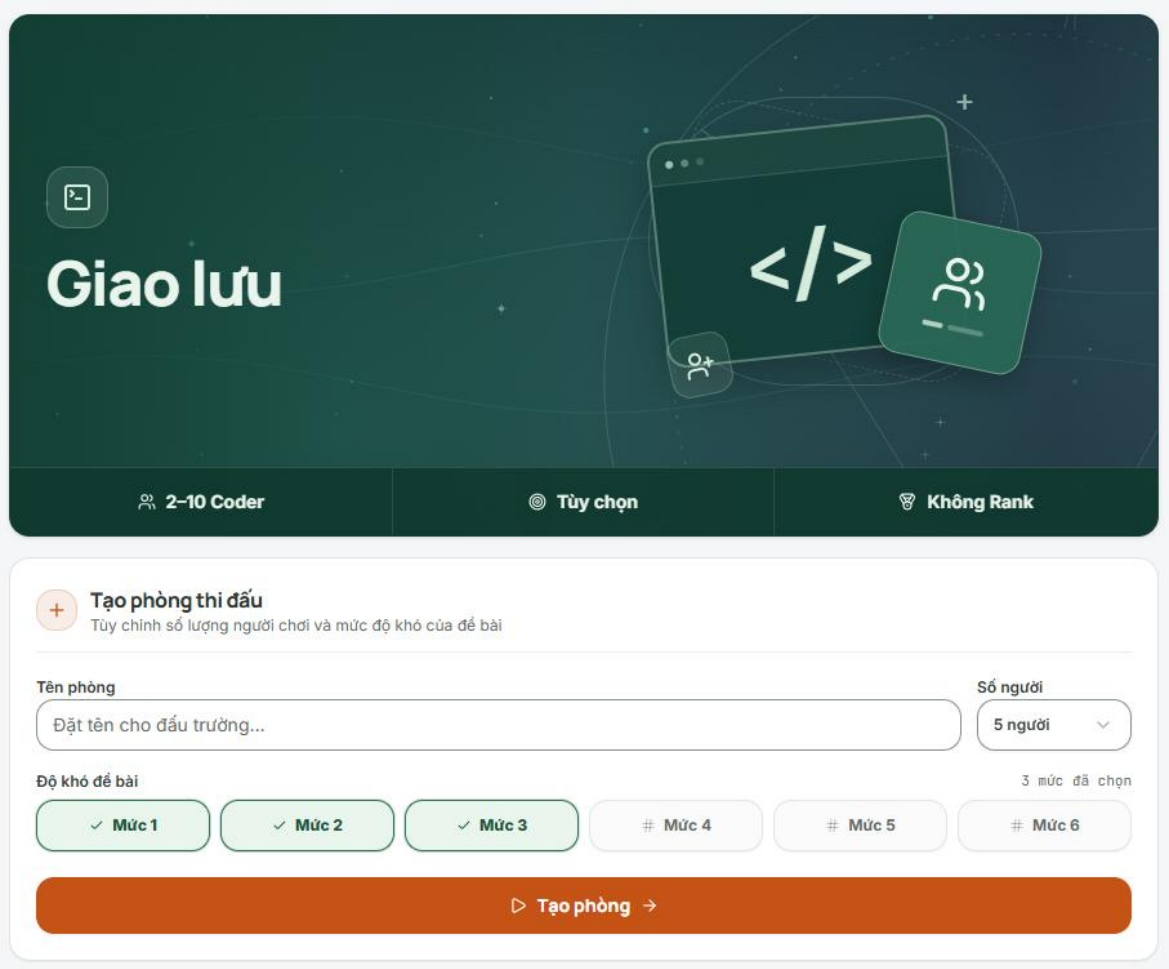
Bên cạnh các chức năng luyện tập và tổ chức cuộc thi thông thường, hệ thống đã tích hợp Code Survival nhằm tạo môi trường thi đấu lập trình nhiều người theo hình thức sinh tồn. Chế độ này kết hợp ghép trận, giải bài theo thời gian giới hạn, chấm bài tự động và loại người chơi qua từng vòng. Toàn bộ trạng thái trận đấu được quản lý tập trung tại Server và đồng bộ đến các Client thông qua WebSocket.

Chức năng lựa chọn chế độ thi đấu: Giao diện Code Survival cung cấp ba chế độ Practice (Giao lưu), Normal (Đấu thường) và Ranked (Đấu xếp hạng). Practice dành cho các trận tùy chỉnh từ 2 đến 10 người, không ảnh hưởng đến điểm xếp hạng. Normal tổ chức trận đấu 5 người và trao điểm kinh nghiệm (EXP) theo kết quả. Ranked cũng có 5 người nhưng sử dụng điểm Rating khi ghép trận và điều chỉnh điểm xếp hạng sau khi trận đấu kết thúc. Giao diện đồng thời hiển thị cấp độ, điểm Rating, bậc Rank và một số thành tích của người chơi.



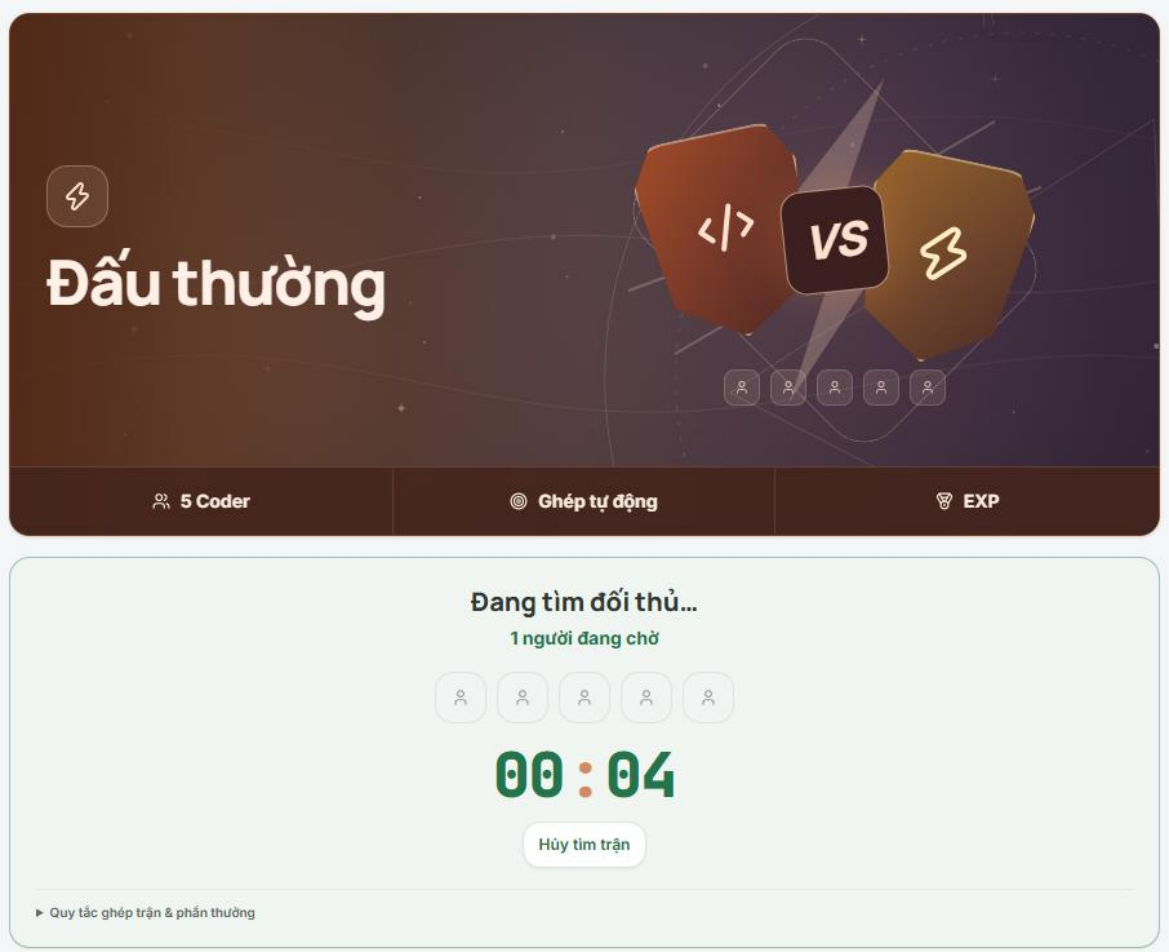
Hình 4.9. Giao diện lựa chọn ba chế độ thi đấu Code Survival

Chức năng tạo phòng và tham gia Practice: Ở chế độ Practice, người chơi có thể tạo phòng với tên và số lượng thành viên tùy chọn. Chủ phòng được thiết lập độ khó của đề, thời gian mỗi vòng và thời gian chung kết trước khi bắt đầu trận đấu. Người khác có thể tham gia thông qua danh sách phòng đang mở, mã phòng hoặc liên kết mời. Trong phòng chờ, hệ thống hiển thị danh sách người tham gia và cho phép chủ phòng thay đổi cấu hình khi trận chưa bắt đầu. Việc bắt đầu trận chỉ được chấp nhận khi đáp ứng điều kiện về số người và nguồn đề phù hợp.



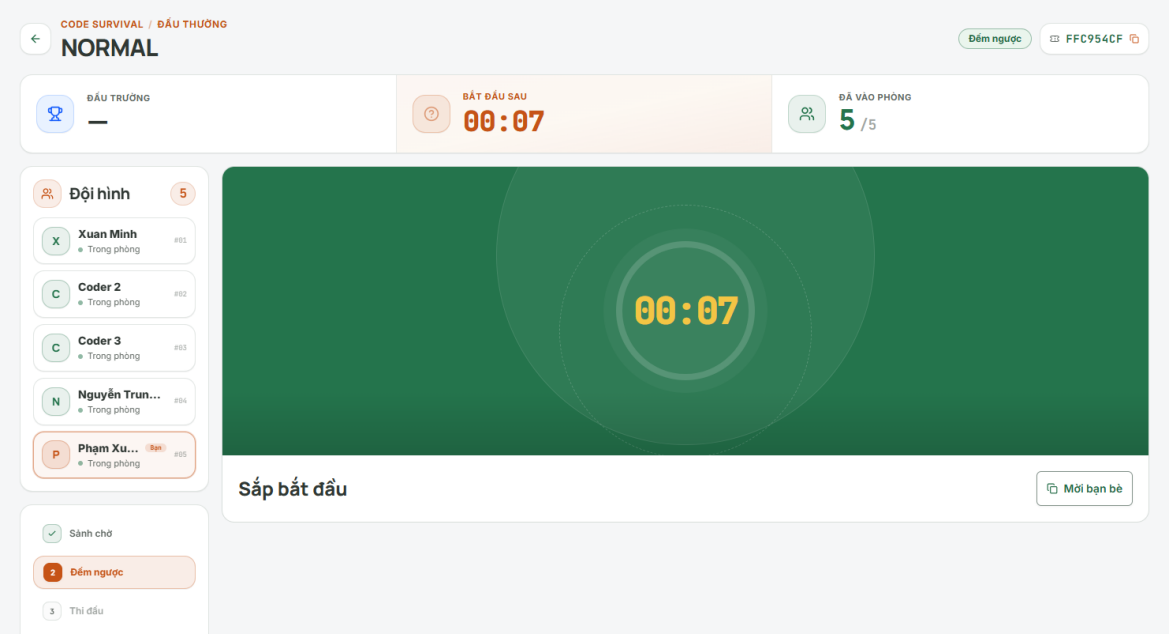
Hình 4.10. Giao diện phòng Practice với cấu hình trận đấu

Chức năng ghép trận tự động: Đối với Normal và Ranked, người chơi được đưa vào hàng đợi để Server tìm đủ năm người tham gia. Chế độ Normal ưu tiên ghép người chơi theo cấp độ trong giai đoạn đầu tìm trận, trong khi Ranked xét khoảng chênh lệch Rating và mở rộng dần phạm vi tìm kiếm theo thời gian chờ. Khi tìm được trận, hệ thống gửi yêu cầu xác nhận đến từng người chơi trong thời gian quy định. Trận đấu chỉ được bắt đầu khi đủ năm người xác nhận; trường hợp từ chối hoặc hết hạn sẽ được xử lý để tránh khởi tạo trận không hợp lệ.



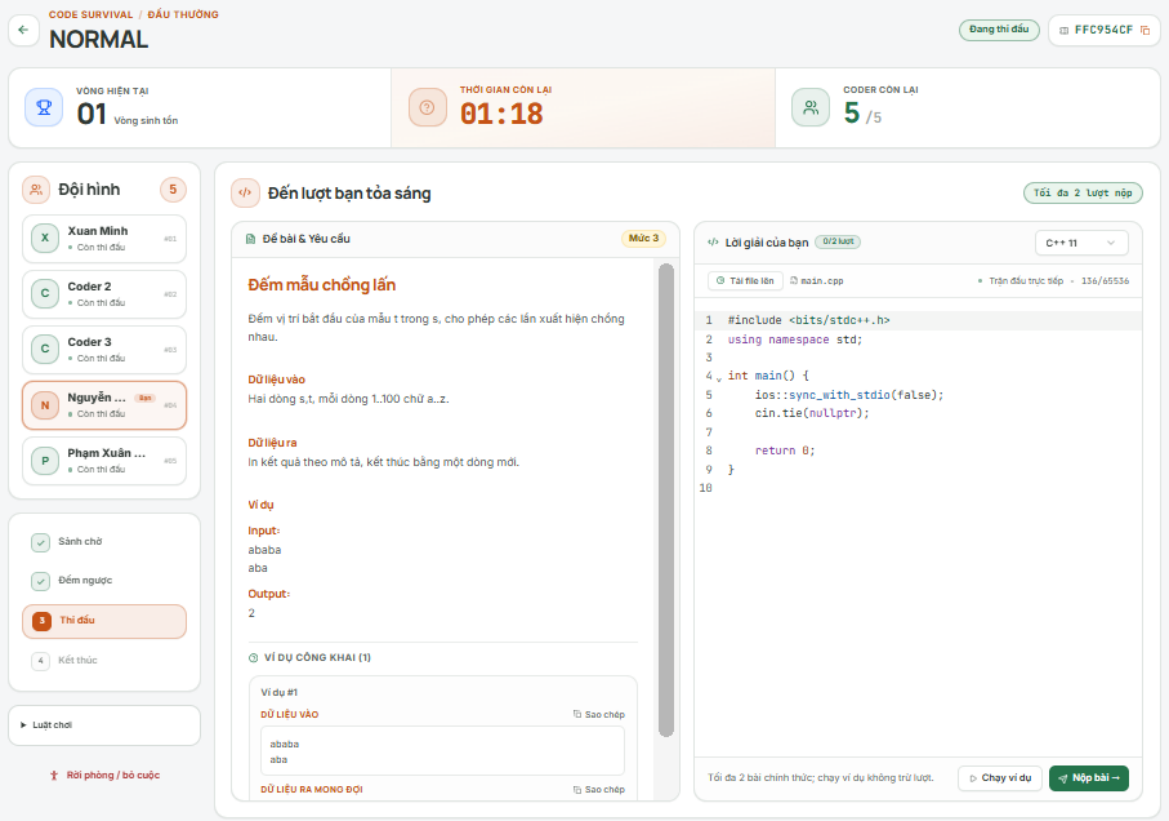
Hình 4.11. Giao diện tìm đối thủ ở chế độ Đấu thường

Chức năng quản lý và đồng bộ trạng thái trận đấu: Sau khi đủ điều kiện bắt đầu, phòng chuyển sang giai đoạn đếm ngược trước khi mở vòng thi. Giao diện hiển thị danh sách người chơi, vòng hiện tại, số người còn lại và thời gian tương ứng. Khi trạng thái trận đấu thay đổi, Server gửi sự kiện để các Client cập nhật thông tin. Nhờ đó, những người tham gia cùng một phòng có thể theo dõi thống nhất tiến trình trận đấu trên các thiết bị hoặc phiên trình duyệt khác nhau. Hệ thống cũng hỗ trợ cập nhật trạng thái người chơi khi rời phòng, bỏ cuộc hoặc mất kết nối.



Hình 4.12. Danh sách người chơi và trạng thái một trận Code Survival

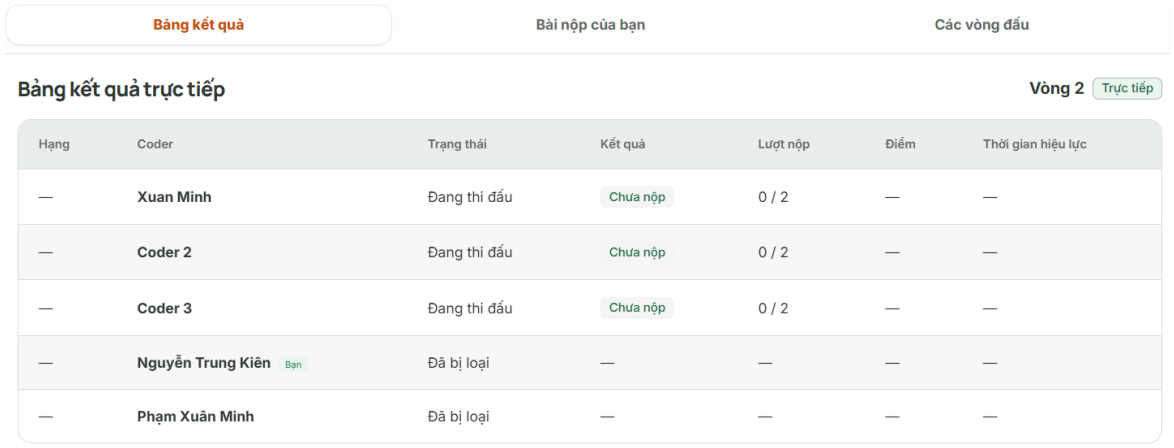
Chức năng giải bài và nộp mã nguồn trong trận: Trong mỗi vòng thi, hệ thống cung cấp một bài lập trình tương ứng với độ khó đã thiết lập. Người chơi giải bài trực tiếp trên Workspace tích hợp, với nội dung đề, trình soạn thảo mã nguồn và đồng hồ đếm ngược. Mỗi người được thực hiện tối đa hai lượt nộp chính thức trong một vòng. Bài nộp được Server tiếp nhận và chuyển đến Worker để chấm; trạng thái chờ chấm, đang chấm và kết quả được cập nhật trên giao diện. Người chơi đã bị loại chỉ có thể tiếp tục theo dõi trận đấu, không được gửi thêm bài nộp cho các vòng tiếp theo.



Hình 4.13. Giao diện giải bài, nộp mã nguồn và theo dõi thời gian trong trận

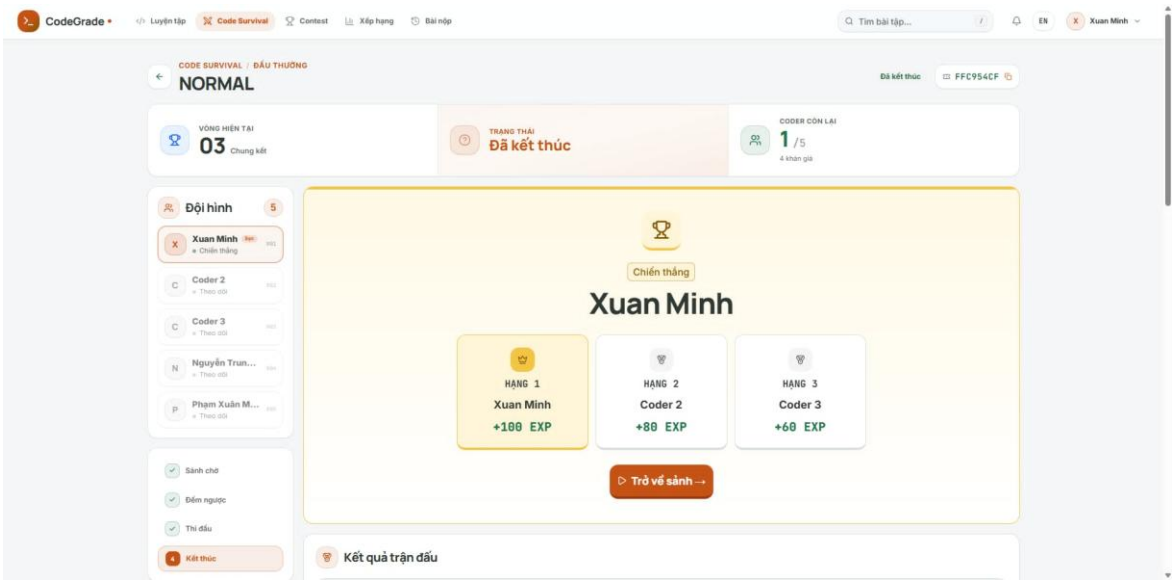
Chức năng đánh giá và loại người chơi qua các vòng: Sau mỗi vòng, hệ thống tổng hợp kết quả bài nộp để xác định thứ hạng và những người được đi tiếp. Người đạt AC được ưu tiên xét theo thời gian hiệu lực, có tính đến số lần nộp sai trước đó. Giao diện hiển thị bảng kết quả, trạng thái của từng người và thông báo đi tiếp hoặc bị loại. Kết quả các vòng đã hoàn thành được lưu lại để người chơi theo dõi diễn biến trận đấu.

Ngoài các vòng thông thường, hệ thống còn hỗ trợ vòng phân định đồng hạng (TIE_BREAK) và vòng cứu hộ (RESCUE) trong những tình huống cần xử lý bổ sung. Trường hợp không thể xác định người chiến thắng hợp lệ, trận đấu có thể kết thúc ở trạng thái chưa phân định kết quả thay vì tự chọn người thắng.



Hình 4.14. Bảng trạng thái người chơi tại vòng 2 của Code Survival

Chức năng tổng kết trận đấu và cập nhật thành tích: Khi xác định được người chiến thắng, hệ thống hiển thị kết quả cuối cùng gồm thứ hạng và thành tích của các thành viên. Với Normal và Ranked, người chơi nhận EXP theo vị trí đạt được; riêng Ranked còn được điều chỉnh Survival Rating và bậc Rank dựa trên kết quả trận đấu. Đối với Practice, kết quả được ghi nhận nhưng không trao EXP hoặc thay đổi Rating. Ngoài ra, hệ thống cung cấp lịch sử thi đấu và bảng xếp hạng Ranked để người dùng xem lại kết quả và theo dõi quá trình phát triển thành tích.



Hình 4.15. Giao diện kết quả trận và điểm kinh nghiệm được trao

Như vậy, Code Survival đã mở rộng hệ thống từ hình thức luyện tập và thi đấu lập trình truyền thống sang môi trường thi đấu sinh tồn theo thời gian thực. Chức

năng này kết hợp quản lý nhiều người chơi, đồng bộ trạng thái qua WebSocket, chấm bài tự động và cập nhật kết quả trên cùng một nền tảng Client–Server.

4.4. Kết quả kiểm thử và đánh giá kỹ thuật

Kết quả thực nghiệm được tổng hợp theo bốn nhóm chính: kiểm thử chức năng và giao thức, đo giao tiếp WebSocket, đánh giá khả năng chấm bài bất đồng bộ và mô phỏng tính công bằng của Code Survival. Các kết quả được sử dụng để kiểm chứng những cơ chế đã thiết kế, đồng thời đánh giá khả năng xử lý của hệ thống trong phạm vi điều kiện thực nghiệm.

Kết quả kiểm thử chức năng và giao thức: Các bộ kiểm thử tự động ghi nhận 107/107 ca Backend và 89/89 ca Frontend đạt yêu cầu. Sáu kịch bản kiểm thử trình duyệt cũng hoàn thành thành công, bao gồm những thao tác liên quan đến luyện tập, quản lý nội dung và Code Survival. Đối với hệ thống chấm bài, cả bảy trường hợp kiểm thử bằng Docker đều trả về kết quả đúng với kỳ vọng. Sáu trường hợp kiểm tra giao thức WebSocket xác nhận hoạt động của các cơ chế xác thực, phân quyền, kết nối lại và kiểm soát yêu cầu trùng lặp. Ngân hàng bài tập được kiểm chứng riêng thông qua các lượt thực thi lời giải tham chiếu.

Bảng 4.2. Tổng hợp kết quả kiểm thử chức năng và giao thức

Nhóm kiểm thử	Kết quả	Nội dung kiểm chứng
Backend	107/107 đạt	Nghiệp vụ, phân quyền, quản lý dữ liệu và quyết toán thành tích
Frontend	89/89 đạt	Logic giao diện, hiển thị và xử lý tương tác
Kiểm thử trình duyệt	6/6 đạt	Các thao tác luyện tập, quản lý nội dung và Code Survival
Chấm bài bằng Docker	7/7 đạt	C++11, C++17, C++20, Python 3.11; kết quả AC, WA, CE, TLE
Giao thức WebSocket	6/6 đạt	Xác thực, phân quyền, kết nối lại và xử lý yêu cầu trùng lặp
Ngân hàng bài tập	2.800/2.800 lượt chạy và 125/125 phép đối chiếu đạt	Kiểm chứng 320 bài tập với 1.600 testcase

Đáng chú ý, các phép kiểm thử nghiệp vụ Code Survival xác nhận việc tính EXP, Rating và quyết toán thành tích cho năm người chơi, bao gồm trường hợp đồng hạng.

Khi thực hiện lại thao tác quyết toán, hệ thống không cộng thưởng trùng lặp. Đối với giao thức WebSocket, yêu cầu được gửi lại với cùng requestId và nội dung chỉ tạo một bản ghi nghiệp vụ, trong khi trường hợp sử dụng lại mã định danh với nội dung khác bị từ chối. Những kết quả này cho thấy các cơ chế kiểm soát dữ liệu hoạt động đúng trong những tình huống đã kiểm tra.

Kết quả đo giao tiếp WebSocket: Các phép đo ghi nhận đầy đủ 12.300 phản hồi ACK và không có ACK lỗi. Thời gian phản hồi p95 có xu hướng tăng theo số lượng Client đồng thời, tuy nhiên vẫn có sự dao động giữa các lượt đo.

Bảng 4.3. Thời gian phản hồi WebSocket theo số Client đồng thời

Số Client	Số ACK mỗi lượt	Khoảng p95 ghi nhận (ms)
1	50	0,994–2,240
10	500	1,940–3,000
30	1.500	5,705–10,707

Ở mức 30 Client, thời gian phản hồi p95 nằm trong khoảng 5,705–10,707 ms. Mặc dù không ghi nhận ACK lỗi, giá trị RTT lớn nhất trong toàn bộ phép đo đạt 1.534,593 ms, cho thấy một số yêu cầu có thời gian phản hồi cao hơn đáng kể. Kết quả xác nhận khả năng tiếp nhận và phản hồi các thông điệp PING trong điều kiện đã khảo sát, nhưng chưa phản ánh đầy đủ độ trễ của những nghiệp vụ thi đấu có xác thực hoặc giao tiếp qua Internet.

Kết quả chấm bài bất đồng bộ: Thực nghiệm hoàn thành 18 lượt thử tải với tổng cộng 96 bài nộp. Cả 96 bài đều đạt Accepted (AC), hoàn thành đủ năm testcase, nhận 100 điểm và được ghi nhận chính xác trong MySQL. Kết quả so sánh thông lượng và thời gian xử lý đầu cuối giữa một và hai Worker được trình bày trong Bảng 4.4.

Bảng 4.4. So sánh kết quả chấm bài với một và hai Worker

Client đồng thời	Thông lượng 1 Worker (bài/s)	Thông lượng 2 Worker (bài/s)	Trung bình p95 E2E 1 Worker (s)	Trung bình p95 E2E 2 Worker (s)
1	0,28411	0,31231	3,431	3,084
5	0,32953	0,53758	15,457	9,203
10	0,31309	0,57128	32,534	17,454

Ở mức một Client, sự khác biệt giữa hai cấu hình Worker tương đối nhỏ. Khi tăng lên năm Client, thông lượng với hai Worker cao gấp khoảng 1,63 lần cấu hình một Worker. Tại mức 10 Client, thông lượng tăng từ 0,31309 lên 0,57128 bài/giây, tương đương 1,82 lần; trung bình p95 thời gian xử lý đầu cuối giảm từ 32,534 xuống 17,454 giây, tương đương 46,35%. Kết quả cho thấy việc bổ sung Worker có thể cải thiện khả năng xử lý khi nhiều bài nộp cùng chờ chấm. Tuy nhiên, mỗi cấu hình chỉ được lặp lại ba lần với cùng một bài Python; các giá trị p95 trong bảng là trung bình p95 của từng lượt thử, không phải phân vị tính trên toàn bộ bài nộp.

Kết quả mô phỏng tính công bằng của Code Survival: Trong 10.000 lượt mô phỏng với 10 người chơi và bảy suất đi tiếp, phương pháp lựa chọn theo thời điểm hoàn thành chấm có 6.151 lượt khác tập tham chiếu, tương ứng tỷ lệ sai lệch 61,51%. Ngược lại, phương pháp sử dụng thời điểm Server tiếp nhận bài không ghi nhận trường hợp sai lệch, tương ứng 0%. Mô phỏng cũng ghi nhận 24.650 trong tổng số 100.000 bài giả lập được tiếp nhận trong thời hạn nhưng hoàn thành chấm sau thời điểm kết thúc vòng.

Sự khác biệt trên cho thấy độ trễ Worker có thể làm thay đổi tập người đi tiếp nếu thứ hạng được xác định theo thời điểm hoàn tất chấm. Việc sử dụng thời điểm Server tiếp nhận bài giúp loại bỏ ảnh hưởng này trong mô hình đã khảo sát, phù hợp với cơ chế xác định kết quả được thiết kế cho Code Survival. Tuy nhiên, tỷ lệ sai lệch 0% chỉ áp dụng cho giả định mỗi người có một bài AC và tập tham chiếu được xác định theo thứ tự tiếp nhận; phép mô phỏng chưa bao quát các tình huống nhiều lượt nộp, đồng hạng, lỗi hạ tầng hoặc mất kết nối trong trận thực tế.

Đánh giá chung: Kết quả kiểm thử cho thấy những chức năng và cơ chế được khảo sát hoạt động đúng trong phạm vi thực nghiệm. Hệ thống tiếp nhận và phản hồi đầy đủ các thông điệp WebSocket ở những mức tải đã thử, xử lý chính xác các bài nộp thông qua Worker và cải thiện thông lượng khi bổ sung Worker. Đối với Code Survival, kết quả kiểm thử nghiệp vụ và mô phỏng cung cấp cơ sở đánh giá các cơ chế quyết toán thành tích, hạn chế xử lý trùng lặp và xác định người đi tiếp theo thời điểm Server tiếp nhận bài.

Các phép đo hiện được thực hiện trên cùng một máy, với số lượng Client và số lượt thử còn giới hạn. Hệ thống chưa được đánh giá đầy đủ bằng thử tải liên tục, đo độ trễ đồng bộ trạng thái giữa nhiều máy hoặc thử tải một trận Code Survival hoàn chỉnh. Vì vậy, các kết luận về hiệu năng và tính công bằng chỉ áp dụng trong những điều kiện đã kiểm chứng.

CHƯƠNG 5. THẢO LUẬN, ĐÁNH GIÁ VÀ ĐÓNG GÓP

5.1. Phân tích và đánh giá kết quả

Đối chiếu với mục tiêu đề tài, hệ thống đã triển khai được các chức năng chính của một nền tảng luyện tập và thi đấu lập trình trực tuyến theo mô hình Client–Server. Các hoạt động quản lý người dùng, luyện tập, chấm bài tự động và tổ chức thi đấu được tích hợp trên cùng một hệ thống, với Server đảm nhiệm xử lý nghiệp vụ và quản lý dữ liệu tập trung. Kết quả kiểm thử Backend, Frontend và các kịch bản tương tác trên trình duyệt đều đạt yêu cầu trong phạm vi khảo sát. Bên cạnh đó, ngân hàng bài tập được kiểm chứng bằng lời giải tham chiếu, tạo cơ sở cho việc sử dụng chung nguồn đề trong luyện tập và thi đấu.

Về giao tiếp mạng, việc sử dụng WebSocket cho phép Client và Server trao đổi yêu cầu, phản hồi và cập nhật trạng thái thông qua kết nối hai chiều. Hệ thống kết hợp mã định danh yêu cầu, lưu phản hồi đã xử lý và đánh số thứ tự sự kiện nhằm hỗ trợ kiểm soát thao tác trùng lặp và khôi phục kết nối. Các phép kiểm thử giao thức xác nhận những cơ chế được khảo sát hoạt động đúng; đồng thời, phép đo với tối đa 30 Client ghi nhận đầy đủ phản hồi ACK. Tuy nhiên, thời gian phản hồi vẫn có sự dao động khi số kết nối tăng, và phép đo PING chưa phản ánh đầy đủ độ trễ của các nghiệp vụ thi đấu thực tế. Kết quả cho thấy khả năng giao tiếp mạng bước đầu đáp ứng yêu cầu của hệ thống trong điều kiện thực nghiệm.

Đối với chức năng chấm bài tự động, việc tách Worker khỏi Server giúp các tác vụ biên dịch và thực thi mã nguồn được xử lý bất đồng bộ, không buộc Server phải chờ hoàn thành quá trình chấm trước khi tiếp nhận yêu cầu khác. Các trường hợp kiểm thử Docker và thử tải đều ghi nhận kết quả chấm chính xác. Đặc biệt, khi tăng từ một lên hai Worker ở mức 10 Client đồng thời, thông lượng xử lý tăng 1,82 lần và trung bình p95 thời gian xử lý đầu cuối giảm 46,35%. Điều này cho thấy cách tổ chức Worker độc lập có khả năng cải thiện hiệu quả xử lý khi nhiều bài nộp cùng chờ chấm, đồng thời tạo cơ sở để mở rộng năng lực chấm bài theo tài nguyên triển khai.

Đối với Code Survival, hệ thống đã kết hợp cơ chế quản lý trận đấu nhiều người với quá trình chấm bài bất đồng bộ và cập nhật trạng thái qua mạng. Các phép kiểm thử nghiệp vụ xác nhận việc tính thành tích, xử lý đồng hạng và hạn chế cộng thưởng trùng lặp trong những trường hợp đã khảo sát. Một vấn đề quan trọng là thời điểm hoàn thành chấm có thể khác thứ tự tiếp nhận bài, ảnh hưởng đến việc xác định người đi tiếp. Kết quả mô phỏng cho thấy phương pháp xếp hạng theo thời điểm Server tiếp nhận bài loại bỏ được sai lệch do độ trễ Worker trong mô hình thử nghiệm, trong khi phương pháp dựa trên thời điểm hoàn thành chấm có tỷ lệ sai lệch đáng kể. Kết quả này hỗ trợ tính hợp lý của cơ chế xác định thứ hạng được áp dụng trong Code Survival, nhưng chưa chứng minh tính công bằng tuyệt đối trong mọi tình huống thi đấu.

Nhìn chung, kết quả triển khai và thực nghiệm cho thấy hệ thống đáp ứng những yêu cầu cốt lõi của đề tài, đặc biệt ở giao tiếp Client–Server, xử lý chấm bài bất đồng bộ, quản lý dữ liệu nhất quán và tổ chức thi đấu trực tuyến. Các kết quả kiểm thử cung cấp cơ sở đánh giá tính đúng đắn của những chức năng đã triển khai và hiệu quả của một số giải pháp kỹ thuật được lựa chọn. Tuy nhiên, khả năng vận hành ổn định ở quy mô lớn và trong điều kiện mạng thực tế vẫn cần được đánh giá thêm trước khi có thể đưa ra kết luận tổng quát về hiệu năng và độ tin cậy của hệ thống.

5.2. Điểm mới và đóng góp của dự án

Điểm nổi bật của đề tài là việc phát triển và tích hợp chế độ Code Survival vào hệ thống luyện tập và thi đấu lập trình trực tuyến. Bên cạnh các chức năng phổ biến như ngân hàng bài tập, chấm bài tự động và tổ chức Contest, Code Survival bổ sung hình thức thi đấu sinh tồn nhiều vòng, trong đó người chơi trực tiếp cạnh tranh quyền đi tiếp thông qua kết quả giải bài lập trình. Hình thức này tạo thêm môi trường giao lưu và thi đấu nhóm nhỏ, đồng thời tận dụng những chức năng nền tảng đã được xây dựng.

Về chức năng, Code Survival được triển khai với ba chế độ gồm Giao lưu, Đấu thường và Đấu xếp hạng. Chế độ Giao lưu hỗ trợ tạo phòng tùy chỉnh để luyện tập cùng bạn bè; Đấu thường và Đấu xếp hạng sử dụng cơ chế ghép trận tự động với

những tiêu chí lựa chọn người chơi khác nhau. Các chế độ được tổ chức trên cùng một quy trình quản lý trận đấu, bao gồm xác nhận tham gia, điều khiển vòng thi, loại người chơi và tổng kết thành tích. Việc sử dụng chung ngân hàng đề và bộ chấm tự động giúp kết nối hoạt động luyện tập với thi đấu trên một nền tảng thống nhất.

Về mặt kỹ thuật, đóng góp quan trọng của Code Survival là cơ chế xác định kết quả trong điều kiện các bài nộp được Worker chấm bất đồng bộ. Thay vì sử dụng thứ tự hoàn thành chấm, hệ thống dựa trên thời điểm Server tiếp nhận bài, kết hợp kết quả Accepted và thời gian phạt từ các lượt nộp sai để xếp hạng. Cơ chế này hạn chế ảnh hưởng của sự chênh lệch thời gian xử lý giữa các Worker đến quyền đi tiếp. Ngoài ra, các tình huống đồng hạng, lỗi chấm bài và không xác định được người chiến thắng hợp lệ được xử lý theo những quy tắc riêng, thay vì tự động đưa ra kết quả khi chưa đủ căn cứ.

Một đóng góp khác là việc kết hợp quản lý trạng thái trận đấu tập trung với giao tiếp WebSocket và cơ chế kiểm soát dữ liệu đồng thời. Server quyết định việc chuyển vòng, xác định kết quả và cập nhật thành tích, trong khi các Client tiếp nhận sự kiện để đồng bộ trạng thái. Quá trình quyết toán EXP và Survival Rating được thực hiện trong giao dịch cơ sở dữ liệu, kết hợp kiểm tra lịch sử phần thưởng nhằm hạn chế cập nhật trùng lặp. Cách tổ chức này đáp ứng những yêu cầu về tính nhất quán khi nhiều người dùng cùng tham gia một trận đấu qua mạng.

Nhìn chung, đóng góp chính của dự án không nằm ở việc phát triển những công nghệ riêng biệt, mà ở việc xây dựng một nền tảng Client–Server tích hợp và phát triển Code Survival thành hình thức thi đấu lập trình sinh tồn nhiều người. Sự kết hợp giữa chấm bài bất đồng bộ, quản lý trạng thái theo thời gian thực và các quy tắc xác định kết quả là điểm khác biệt nổi bật của hệ thống trong phạm vi đề tài.

5.3. Những hạn chế của hệ thống

Mặc dù đã triển khai được các chức năng chính và đạt kết quả kiểm thử trong phạm vi khảo sát, hệ thống vẫn tồn tại một số hạn chế về khả năng mở rộng, phạm vi hỗ trợ và điều kiện vận hành thực tế.

Về hạ tầng, quá trình chấm bài phụ thuộc vào Docker và tài nguyên tính toán của máy triển khai. Việc bổ sung Worker có thể cải thiện khả năng xử lý bài nộp, nhưng hiệu quả còn chịu ảnh hưởng bởi số lượng tác vụ, giới hạn tài nguyên và đặc điểm chương trình được thực thi. Kiến trúc hiện tại sử dụng Server và cơ sở dữ liệu tập trung, chưa có cơ chế dự phòng hoàn chỉnh khi các thành phần này gặp sự cố. Bên cạnh đó, môi trường chấm bài mới hỗ trợ C++11, C++17, C++20 và Python 3.11, nên phạm vi ngôn ngữ lập trình dành cho người dùng còn hạn chế.

Đối với Code Survival, các trận Đấu thường và Đấu xếp hạng yêu cầu đủ năm người tham gia để thực hiện ghép trận. Vì vậy, thời gian chờ có thể kéo dài khi số người tham gia ít hoặc mức chênh lệch trình độ giữa những người trong hàng đợi lớn. Ngoài ra, chất lượng thi đấu phụ thuộc vào nguồn bài tập và bộ testcase được sử dụng. Mặc dù hệ thống có cơ chế kiểm tra cấu hình đề và lời giải tham chiếu, việc bảo đảm testcase bao quát đầy đủ những trường hợp đặc biệt vẫn phụ thuộc vào chất lượng xây dựng đề.

Về phạm vi đánh giá, các phép đo hiệu năng được thực hiện trên cùng một máy với số lượng Client và thời gian thử nghiệm còn giới hạn. Hệ thống chưa được kiểm thử tải liên tục ở quy mô lớn hoặc đo riêng độ trễ đồng bộ trạng thái giữa nhiều Client trên các máy khác nhau. Đối với Code Survival, phép mô phỏng công bằng mới tập trung vào ảnh hưởng của độ trễ Worker trong vòng loại, chưa bao quát đầy đủ những tình huống như nhiều lượt nộp, mất kết nối, đồng hạng và lỗi hạ tầng trong một trận đấu hoàn chỉnh.

Những hạn chế trên cho thấy hệ thống đã đáp ứng các mục tiêu triển khai trong phạm vi đề tài, nhưng khả năng vận hành ổn định và phục vụ số lượng lớn người dùng trong môi trường thực tế vẫn cần được kiểm chứng thêm. Đây cũng là cơ sở để xác định những hướng hoàn thiện và mở rộng hệ thống trong thời gian tới.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

Đề tài "Xây dựng hệ thống luyện tập và thi đấu lập trình trực tuyến theo mô hình Client–Server" đã xây dựng được một nền tảng web tích hợp các chức năng quản lý người dùng, ngân hàng bài tập, soạn thảo mã nguồn, chấm bài tự động và tổ chức thi đấu lập trình. Hệ thống hỗ trợ các vai trò CODER, AUTHOR và ADMIN, đáp ứng nhu cầu luyện tập, quản lý nội dung và theo dõi kết quả trên một nền tảng thống nhất.

Về mặt kỹ thuật, hệ thống được tổ chức theo mô hình Client–Server, sử dụng WebSocket để trao đổi dữ liệu và đồng bộ trạng thái giữa các thành phần. Quá trình chấm bài được xử lý bất đồng bộ thông qua Worker kết hợp Docker Sandbox, giúp phân tách hoạt động thực thi mã nguồn khỏi quá trình xử lý nghiệp vụ của Server. Bên cạnh đó, các cơ chế kiểm soát yêu cầu trùng lặp, quản lý giao dịch và khôi phục kết nối được triển khai nhằm duy trì tính nhất quán dữ liệu khi có nhiều người dùng hoạt động đồng thời.

Kết quả nổi bật của đề tài là việc phát triển và tích hợp Code Survival, bổ sung hình thức thi đấu lập trình sinh tồn nhiều vòng với ba chế độ Giao lưu, Đấu thường và Đấu xếp hạng. Chế độ này kết hợp ghép trận, quản lý trạng thái thi đấu, chấm bài tự động và xác định kết quả theo những quy tắc thống nhất. Đặc biệt, việc sử dụng thời điểm Server tiếp nhận bài làm cơ sở xếp hạng góp phần hạn chế ảnh hưởng của độ trễ chấm bài đến quyền đi tiếp của người chơi.

Thông qua quá trình triển khai và kiểm thử, đề tài đã hiện thực hóa được những yêu cầu cốt lõi về giao tiếp Client–Server, xử lý đồng thời, chấm bài bất đồng bộ và tổ chức thi đấu trực tuyến. Mặc dù vẫn còn giới hạn về quy mô thực nghiệm và khả năng vận hành trong môi trường thực tế, kết quả đạt được tạo nền tảng để tiếp tục hoàn thiện và mở rộng hệ thống. Đồng thời, quá trình thực hiện đề tài giúp nhóm vận dụng kiến thức Lập trình mạng vào việc xây dựng một ứng dụng có nhiều thành phần phối hợp và trao đổi dữ liệu qua mạng.

6.2. Hướng phát triển

Trong thời gian tới, hệ thống có thể tiếp tục được hoàn thiện theo hướng nâng cao khả năng phục vụ nhiều người dùng, mở rộng chức năng luyện tập và cải thiện trải nghiệm thi đấu Code Survival. Các hướng phát triển được xác định dựa trên kiến trúc hiện tại và những vấn đề cần tiếp tục nghiên cứu trong quá trình triển khai thực tế.

Về hạ tầng và giao tiếp mạng, hệ thống có thể mở rộng số lượng Worker để tăng năng lực chấm bài, kết hợp cơ chế giám sát hàng đợi, trạng thái tác vụ và tài nguyên thực thi nhằm điều phối công việc hiệu quả hơn. Bên cạnh đó, cần bổ sung giải pháp sao lưu, phục hồi dữ liệu và giám sát hoạt động của các dịch vụ. Việc kiểm thử trên nhiều máy qua mạng thực tế, với số lượng Client lớn hơn và thời gian hoạt động kéo dài, cũng là hướng cần thực hiện để đánh giá khả năng mở rộng và độ ổn định của hệ thống.

Đối với chức năng luyện tập, ngân hàng bài tập có thể được mở rộng về số lượng, chủ đề và mức độ khó, đồng thời bổ sung các ngôn ngữ lập trình như Java và JavaScript thông qua việc phát triển môi trường thực thi tương ứng cho Worker. Quy trình kiểm tra đáp án mẫu và testcase cũng có thể được cải tiến nhằm hỗ trợ người tạo đề phát hiện những trường hợp kiểm thử chưa đầy đủ trước khi công bố bài tập.

Đối với Code Survival, hướng phát triển quan trọng là hoàn thiện cơ chế ghép trận theo số lượng người đang chờ, thời gian tìm trận và mức chênh lệch trình độ giữa các đối thủ. Hệ thống cũng có thể cải thiện giao diện theo dõi trận đấu, lịch sử các vòng thi và thống kê thành tích để người chơi dễ dàng đánh giá quá trình thi đấu. Ngoài ra, cần xây dựng thêm các kịch bản kiểm thử nhiều Client tham gia cùng một trận, kết hợp mô phỏng mất kết nối, chấm bài chậm và xử lý đồng thời nhằm đánh giá toàn diện hơn khả năng đồng bộ trạng thái và xác định kết quả.

Những hướng phát triển trên tập trung vào việc hoàn thiện kiến trúc Client–Server, nâng cao hiệu quả xử lý và chất lượng tổ chức thi đấu. Đây là cơ sở để hệ thống từng bước đáp ứng nhu cầu sử dụng trong môi trường có số lượng người học và người tham gia thi đấu lớn hơn.

TÀI LIỆU THAM KHẢO

[1] IETF – *RFC 6455: The WebSocket Protocol*. Chuẩn giao tiếp WebSocket hai chiều giữa client, server và worker.

[2] Spring – *Spring Boot 3.5 Documentation*. Tài liệu xây dựng, cấu hình và vận hành ứng dụng backend.

[3] Spring Framework – *WebSocket API; Data Access with JDBC*. Tài liệu xử lý kết nối WebSocket và truy cập cơ sở dữ liệu.

[4] Spring Security – *Password Storage*. Tài liệu băm và kiểm tra mật khẩu bằng BCrypt.

[5] Oracle – *Java SE 21: WebSocket API*. Tài liệu xây dựng WebSocket client cho worker chấm bài.

[6] Docker – *Docker Compose, Resource Constraints và Docker Engine Security*. Tài liệu triển khai các dịch vụ, giới hạn tài nguyên và bảo mật container chạy bài nộp.

[7] Oracle – *MySQL 8.4 Reference Manual*. Tài liệu cơ sở dữ liệu quan hệ, truy vấn SQL và giao dịch.

[8] Redgate – *Flyway Documentation*. Tài liệu quản lý phiên bản và khởi tạo cấu trúc cơ sở dữ liệu.

[9] React – *React Documentation*. Tài liệu xây dựng giao diện bằng component, quản lý trạng thái và xử lý sự kiện.

[10] Vite – *Vite Guide*. Tài liệu cấu hình môi trường phát triển và đóng gói frontend.

[11] Tailwind CSS – *Tailwind CSS Documentation*. Tài liệu xây dựng giao diện và bố cục thích ứng.